

SOFTWARE REQUIREMENTS SPECIFICATION

Circular Fashion & Upcycling Marketplace

1. Introduction

1.1 Project Context

Lately, everyone is interested in sustainable fashion, but there is still a gap in the apps we use. Popular apps like Vinted or Depop are great for buying and selling second-hand clothes, but they don't really focus on the "life" of the garment. They don't have tools to document upcycling, they don't offer mentorship for new creators, and they don't show the real environmental impact of each item. Our platform is built to fix this. We combine a marketplace with a social community and a "carbon calculator" to make fashion truly circular.

1.2 Problem Statement

The fashion industry is a major polluter. While second-hand apps help, they still have big problems:

- They don't reward people who repair or modify clothes (upcycling).
- They don't provide real data on how much water or CO2 is saved in each transaction.
- New creators don't have a way to learn from experts.
- Users who want to "Swap" clothes don't have a fair way to balance the price if the two items have different values.

1.3 Proposed Solution

Our platform is a web app designed for the circular fashion economy. It offers three main ways to trade: Buying, Selling, and Swapping. What makes us unique is the "Upcycling Track," where creators can post "before and after" photos of their work. We also included a carbon-footprint calculator to show the impact of every deal. To keep users engaged, we use "Eco-Credits" as a virtual currency and a mentor-mentee system for beginners. Everything is kept safe using a simulated escrow payment system and a smart trust-score for users.

1.4 Project Scope

This SRS covers all the requirements. The scope includes: user profiles and roles, item listings, swap and negotiation workflows, simulated payments, social features, and admin tools.

Out of scope: We will not include real banking APIs (payments are simulated) and we are not building a mobile app yet; our focus is on the web application.

1.5 Main Challenges

- How to "lock" an item instantly if it's listed for both sale and swap to avoid double-selling.
- Creating a trust-score system that is fair and hard to cheat.
- Making the carbon-footprint math accurate but also easy for users to understand.
- Managing complex swaps where users trade multiple items at once.
- Building a moderation system that uses both automated filters and human review.

1.6 Completeness Criteria

The project is considered complete when: (a) All 35 functional requirements (Must, Should, and Could) are fully working and tested. (b) All non-functional requirements (like speed and security) meet the goals we set.

2. Overall Description

2.1 Proposed Solution

The application is organised around four core modules: (1) the User & Impact Module, which manages multi-role profiles and sustainability metrics; (2) the Inventory & Upcycle Module, which handles item states and transformation documentation; (3) the Exchange & Negotiation Module, which manages swap logic, bargaining, and price balancing; and (4) the Logistics & Trust Module, which oversees payment simulation, shipping coordination, and peer review. These four modules together fulfil all 35 implemented functional requirements and are supported by a system administration layer.

2.2 Proposed Workflow

The core transaction workflow follows five stages: (1) Discovery — users browse or search the filtered marketplace; (2) Initiation — a Buy, Sell, or Swap request is created; (3) Negotiation — parties may bargain or counter-offer within defined thresholds; (4) Agreement — both parties lock the deal, triggering escrow capture and inventory lock; (5) Fulfilment — item is shipped, delivery confirmed, escrow released, trust scores updated, and Eco-Credits distributed. The Community layer (Style Boards, Drop Notifications, Mentor-Mentee interactions) enriches the platform without entering the transactional flow.

2.3 Stakeholder Analysis

2.3.1 Stakeholder Identification — JBA Framework

Stakeholders were identified by applying the JBA five-lens framework:

- System Goals lens: Who benefits directly if the platform succeeds? → Registered Users, Upcyclers, Marketing Team.
- Affected Parties lens: Whose behaviour or livelihood is changed by the system? → All user classes, Shipping provider, Payment Gateway, Consumer Protection Authority.
- Decision-Makers lens: Who controls scope, priorities, and deployment? → System Admin, Community Moderator, Project Team.
- Subject Matter Experts lens: Who provides domain knowledge? → Upcyclers (craft expertise), Data Analyst (trend data), Technical Support.
- Regulators lens: Who enforces compliance standards? → Consumer Protection Authority, Environmental Organisations.

This analysis yielded twelve stakeholder groups, described and classified below.

2.3.2 Power / Interest Matrix



Quadrant	Stakeholder(s)	Strategy	Rationale
Manage Closely (High Power, High Interest)	System Admin · Payment Gateway · Marketing Team · Community Moderator	Maintain continuous engagement and involvement in all key decisions and requirement reviews.	These parties either control the system or drive its commercial success. Their satisfaction is critical to platform viability.
Keep Satisfied (High Power, Low Interest)	Consumer Protection Authority · Shipping Provider	Meet all legal obligations proactively; communicate compliance updates periodically.	Regulatory body with authority to halt operations but typically uninvolved in day-to-day product decisions.
Keep Informed (Low Power, High Interest)	Registered Users (Buyers / Sellers / Swappers) · Upcyclers	Share release notes, feature changes, and policy updates. Collect feedback through surveys and analytics.	Primary users of the platform with high day-to-day interest but limited power over product roadmap.
Monitor (Low Power, Low Interest)	Guests · Environmental Organizations · Technical Support · Data Analyst	Minimal engagement; inform of major changes only. Review their status periodically in case it shifts.	Low current involvement but may increase in importance as the platform scales.

2.3.3 Internal vs. External Classification

Stakeholder	Classification	Notes
Guest	External	Unauthenticated visitor; no account or stored profile.
Registered User (Buyer/Seller/Swapper)	External	Core platform participants; role determined by action taken, not fixed.
Upcycler	External	Elevated Registered User with verified creator credentials and additional features.
System Admin	Internal	Manages platform operations, approvals, RBAC, and dispute escalations.
Community Moderator	Internal	Reviews flagged content; major decisions require Admin approval.
Data Analyst	Internal	Accesses anonymized platform data to generate trend and sustainability reports.
Payment Gateway	External (Partner)	Third-party financial interface for escrow simulation and payout processing.
Shipping Provider	External (Partner)	Logistics partner for label generation, route optimization, and delivery tracking.
Technical Support	Internal	Diagnoses platform issues; escalates critical failures.
Consumer Protection Authority	External (Regulator)	Government body enforcing consumer rights in e-commerce.
Marketing Team	Internal	Manages promotions, commission modifiers, and newsletter campaigns.
Environmental Organizations	External	Third-party validators of sustainability claims; potential integration partners.

2.4 Design and Implementation Constraints

- **Payment Security:** The system must not save real credit card info. This keeps everything safe and follows official security rules.
- **Content Safety:** Every image a user uploads must be checked for bad content before it is saved.
- **Inventory Accuracy:** The system must track items perfectly so two people cannot book the same item at the same time.
- **Role Changes:** Users cannot upgrade their own accounts (like becoming an "Upcycler"). An Admin must manually approve this.
- **Carbon Math:** The formula used to calculate carbon footprints must be simple and easy for users to understand on the screen.

3. Functional Requirements

3.a System Requirements Specification

Each function below has its own table showing: the Function ID, Name, MoSCoW priority, User Requirement, System Requirements (numbered), and Rationale. Won't Have functions are not included here.

Function ID	F-01
Function Name	Sustainable identity verification
MoSCoW Priority	SHOULD HAVE
User Requirement	The system should allow sellers to submit credentials for verification and should display a trust badge on verified seller profiles.
System Requirements	1.1 The system will record a Verification Request including: the seller's ID, the type of proof (Expert Upcycler or Eco-Verified), the document image, the status (Waiting / Approved / Rejected), the time sent, and which admin checked it. 1.2 If an admin approves it, the badge will show on the seller's profile and on all their items for sale. If it is rejected, the system will send a note to the seller explaining why. 1.3 This proof must be checked every year. 30 days before the year ends, the badge will say Expiring. If the seller does not send new proof, the badge will be taken away.
Rationale	<i>Buyers are more likely to buy from sellers they trust. Without verification, eco-claims are unverifiable and the marketplace is indistinguishable from generic secondhand apps.</i>

Function ID	F-02
Function Name	Carbon Footprint Calculator
MoSCoW Priority	MUST HAVE
User Requirement	The system must calculate and display the estimated CO2 emissions avoided and water saved for each completed transaction, based on the garment material and weight.
System Requirements	2.1 The system will keep a list of materials (like Cotton or Polyester). Each material will have a specific "savings score" for CO2 and water based on official environmental studies. 2.2 When a sale is finished, the system does the math: CO2 Saved = Item Weight × Material CO2 Score; Water Saved = Item Weight × Material Water Score (using the weight and material type the seller provided). 2.3 The computed values shall be persisted in the transaction record and displayed on the transaction confirmation screen, the seller's performance dashboard, and the platform's aggregate sustainability report.
Rationale	<i>Sustainability metrics are the core value proposition of the platform. Without quantifiable environmental impact data, it cannot differentiate itself from generic resale apps. Buyers and</i>

	<i>sellers need tangible evidence of their eco-contribution, and this calculator converts abstract sustainability goals into concrete, shareable numbers — driving engagement and retention.</i>
--	--

Function ID	F-03
Function Name	Multi-Role Permission Controller
MoSCoW Priority	MUST HAVE
User Requirement	The system must allow users to act as a buyer, seller, and swapper at the same time using one account.
System Requirements	3.1 The system will have a list of roles (like Guest, Buyer, Seller, Swapper, Upcycler, Junior Moderator, Lead Moderator, and SuperAdmin). One user account can hold any combination of these roles at once. 3.2 Every time a user tries to do something special (like deleting a post or changing a price), the system must first check if they have the permission to do it. If they aren't allowed, the system will block them and show a "Forbidden" error message. 3.3 When a user switches from one role to another, they don't need to log in again. The system remembers which role they are currently using every time they click a button or send a request.
Rationale	<i>A single person might want to look for clothes (Shopper), list an item for sale (Merchant), and trade an item (Swapper) all at the same time. If the system only allowed one role at a time, it would be too strict or too confusing. This setup makes sure every user has the right access for whatever they are doing at that moment.</i>

Function ID	F-04
Function Name	Preference-Based "Style Feed" Engine
MoSCoW Priority	SHOULD HAVE
User Requirement	The system should present each user with a personalized marketplace feed based on their size, style preferences, and upcycle intensity preference.
System Requirements	4.1 The system will look for items that match the user's size, style, and preferred "upcycle level" (on a scale of 1 to 5). It only shows items that are currently Available. 4.2 Items are ranked by a Match Score: (Style match = 40%) + (Size match = 40%) + (Upcycle level match = 20%). If items have the same score, the newest ones are shown first. 4.3 The list is saved for 15 minutes. If the user clicks "Refresh Feed," the system will clear the old list and look for new items immediately.
Rationale	<i>If it is hard to find items, people will stop using the app. A custom feed helps users find clothes they love quickly, which leads to more sales.</i>

Function ID	F-05
--------------------	------

Function Name	Eco-Credit Reward System
MoSCoW Priority	COULD HAVE
User Requirement	The system could award virtual Eco-Credits to users for sustainable platform behaviours and allow credits to be redeemed against platform fees.
System Requirements	5.1 Every user could have a virtual "Eco-Wallet" to keep track of their points. The points balance can never be less than zero. 5.2 Points shall be given automatically for "Green Actions": Listing an upcycled item: +5 points; Completing a swap deal: +10 points. 5.3 Users can use their points at checkout to get a discount on the platform fee. For example, every 100 points can give a small percentage off the fee.
Rationale	<i>Gamifying the app with points makes it more fun and rewards people for being eco-friendly. It creates a "loyalty loop" where users want to keep using the platform to earn and spend their points.</i>

Function ID	F-06
Function Name	Trust-Score Algorithm
MoSCoW Priority	SHOULD HAVE
User Requirement	The system should display a publicly visible trust score for every user, updated after each transaction.
System Requirements	6.1 The Trust Score is made of three parts: Accuracy (40%), Shipping Speed (30%), and Finished Trades (30%). It uses the average of the last 50 rated deals. 6.2 Accuracy comes from buyer ratings. Shipping is based on how many items were sent on time. Completion is based on how many swaps were actually finished. 6.3 The score is updated immediately after a rating is given. It shows on the profile and item pages as a number out of 100 with a label: Excellent (85+), Good (70–84), Fair (50–69), or New (if the user has fewer than 5 ratings).
Rationale	<i>Trust is the most important part of buying from other people. A clear score gives buyers the information they need to feel safe and encourages sellers to do a great job.</i>

Function ID	F-07
Function Name	Closet Virtualization
MoSCoW Priority	COULD HAVE
User Requirement	The system could allow users to add items to a private digital closet and convert them to active listings with a single action.
System Requirements	7.1 The system may provide a private space where users can upload photos and details of their clothes (up to 200 items) without showing them to the public.

	7.2 If the user decides to sell an item, they could simply click "Publish." The system will then take all the saved info and move the user to the "Condition Assessment" step to finish the post. 7.3 Items in the Virtual Closet is intended to remain hidden from all other users and admins until the owner decides to publish them.
Rationale	<i>A digital closet makes it easier for users to manage their wardrobe. They can "pre-save" their items and list them for sale only when they are ready, which helps increase the number of items on the platform.</i>

Function ID	F-08
Function Name	Upcycle Transformation Logger
MoSCoW Priority	MUST HAVE
User Requirement	The system must allow sellers to attach before-images, after-images, and a structured list of modifications performed when listing a transformed garment.
System Requirements	8.1 Every transformed item gets its own "Change Record." This record must include: at least 1 (and up to 5) "before" pictures and at least 1 (and up to 5) "after" pictures, a list of what was changed, what materials were used, a description of the work, and the exact time it was created. 8.2 Once a manager approves the item, it will show a "Transformed Badge" on its page. Buyers can click a small section to see the "before and after" photos and the list of changes. 8.3 Once the item is live on the site, the "Change Record" cannot be edited. If someone tries to change it later, the system will block them and show an error.
Rationale	<i>Transparency about upcycling modifications builds buyer trust and justifies premium pricing. Without documented transformation history, 'upcycled' claims are unverifiable, undermining platform integrity and exposing it to greenwashing accusations.</i>

Function ID	F-09
Function Name	Advanced Material Taxonomy
MoSCoW Priority	SHOULD HAVE
User Requirement	The system should provide a detailed list of fabric types to help users categorize and filter clothing accurately.
System Requirements	9.1 The system shall organize materials into a 3-level list: Category (e.g., Natural) → Type (e.g., Cotton) → Specific Fabric (e.g., Organic Cotton). 9.2 When listing an item, the seller must pick a specific fabric from this list. This choice will be used to calculate the "Carbon Footprint" and help buyers find exactly what they are looking for. 9.3 The search filters shall be smart; if a buyer filters by "Natural," the system shall show all items made of Cotton, Wool, or any other natural fabric.

Rationale	<i>Eco-friendly buyers care about what their clothes are made of. A detailed fabric list makes searching more precise and ensures that the carbon footprint calculation is as accurate as possible.</i>
------------------	---

Function ID	F-10
Function Name	Condition Assessment Assistant
MoSCoW Priority	MUST HAVE
User Requirement	The system must guide sellers through a mandatory, step-by-step questionnaire to describe the item's condition before they can publish a listing.
System Requirements	10.1 The system shall use a smart UI that asks different questions based on the clothing category (Tops, Shoes, Accessories, etc.). For example, it will only ask about "sole wear" if the seller is listing footwear. 10.2 The assessment shall create a structured "Condition Report" that includes: an overall grade (Excellent, Good, Fair, or Poor), a list of specific defects (type, location, and how bad they are), and a time record of when the check was done. 10.3 The "Publish" button shall stay disabled until the seller finishes all the steps. If a seller tries to post without completing the report, the system shall show an error message.
Rationale	<i>Vague or unclear descriptions are the main cause of fights between buyers and sellers. By forcing a structured check, we remove guesswork and set clear expectations, which reduces returns and builds more trust on the platform.</i>

Function ID	F-11
Function Name	Dynamic Inventory Sync
MoSCoW Priority	MUST HAVE
User Requirement	The system must make sure that the same item cannot be sold and swapped at the same exact time.
System Requirements	11.1 Every item in the system shall have one of four statuses: Available, Reserved, Sold, or Swapped. The system shall protect this status in the database to prevent errors if two users click at the same second. 11.2 When a user starts a checkout or a swap deal, the system shall check if the item is "Available." If it is, the system shall change it to "Reserved." If the item is already taken, the system shall reject the new request and tell the user why. 11.3 Every time an item's status changes, the system shall save a permanent record (Audit Log) showing who did the change and when. This record cannot be edited and will be used to solve any future disputes.
Rationale	<i>Allowing the same item to be sold and swapped simultaneously is a huge failure that destroys trust in a marketplace. Using an automatic "locking" system prevents this mistake and ensures that every deal is safe.</i>

Function ID	F-12
Function Name	Content Moderation Filter
MoSCoW Priority	MUST HAVE
User Requirement	The system must automatically screen all new listings against a prohibited-items list and flag misleading eco-claims before publication.
System Requirements	12.1 Every time someone tries to post an item, the system scans the title, description, and tags. It looks for "bad words" or banned items using a list set by managers. 12.2 If a seller says an item is "organic cotton," the system checks it against an official database. If it doesn't match, the item is marked as a "Fake Claim." 12.3 Flagged posts are sent to a "Pending Review" list for a human to check. The seller will get a message explaining why their post was flagged.
Rationale	<i>Without automated moderation, prohibited items and false sustainability claims would erode user trust and expose the platform to legal liability. Automated first-pass filtering reduces the moderation burden on human admins.</i>

Function ID	F-13
Function Name	Bulk Listing Management
MoSCoW Priority	COULD HAVE
User Requirement	The system may allow professional sellers to create "Collections" and update many items at the same time.
System Requirements	13.1 The system could allow sellers to group their items into a "Collection" (like a Summer Collection) and give them shared details (like the same material or shipping price). 13.2 Sellers may use a "Bulk-Apply" tool to update all items in a collection at once. If any error happens during the update, the system will stop the process to avoid mistakes. 13.3 These collections could be displayed on the seller's profile as a special set, making it easier for buyers to browse their entire new "Drop."
Rationale	<i>Professional designers (Upcyclers) often release many items at once. Bulk tools save them from entering the same data over and over, making the platform more attractive to high-volume sellers.</i>

Function ID	F-15
Function Name	P2P Swap Request Logic
MoSCoW Priority	MUST HAVE

User Requirement	The system must allow any registered user to propose a swap of one or more of their items in exchange for another user's listed item.
System Requirements	15.1 A "Swap Request" includes a list of 1 to 5 items being offered, the item being requested, and a status (like "Waiting," "Accepted," or "Declined"). Requests expire after 72 hours if nothing happens. 15.2 Once a trade is proposed, the items are marked as "Reserved" so no one else can buy them. If an item is already gone, the system will block the trade. 15.3 Both people will get a notification on their phone within 5 seconds of any change (like a new offer or an accepted trade).
Rationale	<i>Swapping is the platform's primary differentiator from standard resale platforms. Without a robust P2P swap request engine, the core sustainability mission — reducing consumption by reusing existing garments — cannot be fulfilled.</i>

Function ID	F-16
Function Name	Swap "Value Balancer" Algorithm
MoSCoW Priority	MUST HAVE
User Requirement	The system shall notify both parties when the estimated value difference between swapped items exceeds a defined threshold, and shall suggest a cash top-up amount.
System Requirements	16.1 When a trade is proposed, the system subtracts the value of one side from the other to find the difference. By default, "unfair" means a difference of more than 20%. 16.2 If the difference is too big, the system adds a "Cash Top-Up" suggestion to the offer. It tells both people exactly how much cash should be added and who should pay it. 16.3 This suggestion is just advice. Both people can choose to ignore it and trade anyway. If they agree to the cash payment, the system handles the extra money safely.
Rationale	<i>People often reject trades because they feel they are losing money. By suggesting a way to "even out" the deal with a little cash, more trades will actually happen. Value imbalance is the primary reason swap proposals are rejected.</i>

Function ID	F-17
Function Name	Multi-Item Counter-Offer Logic
MoSCoW Priority	SHOULD HAVE
User Requirement	The system should allow users to negotiate a swap by changing the items in the bundle or the cash amount offered.

System Requirements	17.1 If a user receives a swap offer they don't like, they can send a "Counter-Offer." They can add or remove items from the deal or change the "cash top-up" amount. 17.2 Once a counter-offer is sent, the system shall release the items from the old offer and "lock" the new items. If any item becomes unavailable during the process, the system shall notify the user. 17.3 To keep things moving, the system shall limit the negotiation to 10 rounds per swap. If no agreement is reached by then, the deal is closed, and users can start a fresh one if they wish.
Rationale	<i>Most swap deals aren't accepted from the first time. Giving users the ability to "bargain" and change the bundle makes it much more likely that they will reach an agreement they are both happy with.</i>

Function ID	F-18
Function Name	Bargaining Interface
MoSCoW Priority	SHOULD HAVE
User Requirement	The system should allow sellers to set automatic price limits to accept or decline offers instantly.
System Requirements	18.1 Sellers can set two secret price limits for their items: an "Auto-Accept" price and an "Auto-Decline" price. 18.2 The system shall handle offers automatically: if a buyer offers a high price (above the Auto-Accept limit), the deal is instantly accepted; if the offer is too low (below the Auto-Decline limit), the system instantly rejects it; if the price is in the middle, the seller is notified to review it manually. 18.3 The system shall keep a record of these automatic actions so sellers can see how people are bidding and adjust their prices if needed.
Rationale	<i>Manual bargaining takes too much time for busy sellers. Automatic limits save time while still giving buyers the chance to "negotiate" for a better price, which leads to more sales.</i>

Function ID	F-19
Function Name	Swap Agreement Lock-in
MoSCoW Priority	MUST HAVE
User Requirement	The system shall prevent both users from changing the item's description or condition once they have both agreed on a swap deal.
System Requirements	19.1 The system shall create a "Swap Agreement" record that saves the IDs of both users, the exact time they signed, and a "Digital Snapshot" of all the item's details at that moment. 19.2 Once both users sign, the system shall lock the items. Any attempt to edit the description, photos, or condition shall be blocked by the system automatically.

	19.3 This "Digital Snapshot" shall be kept in a permanent record. If a dispute happens later, the admin can use it to verify that no one changed the item's details after the deal was made.
Rationale	<i>Changing item details after an agreement is a type of fraud. Locking the descriptions after signing protects both parties and ensures that everyone gets exactly what they agreed on.</i>

Function ID	F-20
Function Name	Expiration Timer Logic
MoSCoW Priority	MUST HAVE
User Requirement	The system shall automatically cancel any pending swap or price offer if the other person does not respond within a specific time limit.
System Requirements	20.1 The system shall check every 60 minutes for any pending swap offers or price bids that have passed their expiration time. 20.2 When an offer expires, the system shall automatically: change its status to "Expired," make the items available for others again, record the event, and send an alert to both users. 20.3 The expiration time shall be controlled by admins (Example: 72 hours for swaps, 48 hours for prices). The system shall clearly show a countdown timer to both users so they know when the offer will end.
Rationale	<i>Leaving offers "pending" forever locks up the seller's clothes and makes other buyers frustrated. Automatic expiration keeps the marketplace moving and ensures that items are always available for active users.</i>

Function ID	F-21
Function Name	Local "Swap Meet" Finder
MoSCoW Priority	COULD HAVE
User Requirement	The system could help users find others nearby for in-person swaps, if they choose to share their location.
System Requirements	21.1 The system may allow users to opt-in to location sharing. If they agree, the system will save their general location to find nearby swap matches. 21.2 Users could search for other swappers within a certain distance (e.g., 25 km to 100 km) to meet and trade items face-to-face. 21.3 To protect privacy, the system is intended to encrypt location data and will never show a user's exact address, only a general area.
Rationale	<i>Meeting in person for a swap is the most eco-friendly way to trade because it completely removes shipping costs and emissions. It also helps build a real-world community around the brand.</i>

Function ID	F-22
Function Name	Escrow Payment Simulation
MoSCoW Priority	MUST HAVE
User Requirement	The system shall hold the buyer's money in a "Virtual Vault" and only release it to the seller after the buyer confirms they have received the item.
System Requirements	22.1 The system shall follow a step-by-step process for every payment: Money Held → Item Shipped → Delivery Confirmed → Money Released. If there is a problem, the status changes to Disputed until an admin decides to release or refund the money. 22.2 Each step must be triggered by an action: The seller scans the shipping label, or the buyer clicks "Item Received." If the buyer forgets to click, the system can automatically release the money after a set time (e.g., 72 hours after delivery). 22.3 Every change in the payment status shall be saved in a permanent "Audit Log." To ensure safety, no one (not even the user) can manually change these money records outside of the official process.
Rationale	<i>Escrow is essential for trust. Buyers feel safe knowing their money won't go to the seller unless they get the item, and sellers know the money is already there. This "Safety Vault" is what makes users trust a new marketplace.</i>

Function ID	F-24
Function Name	Shipping Label Generation Logic
MoSCoW Priority	SHOULD HAVE
User Requirement	The system should create a simulated shipping label and a tracking number for every order that needs to be shipped.
System Requirements	24.1 Once a transaction is confirmed, the system shall calculate the shipping cost based on the item's weight and size. These rates can be adjusted by the admin. 24.2 The system shall generate a "Shipping Label" (PDF) that includes a unique tracking number, the carrier's name, and the expected delivery date. The label will also have a QR code. 24.3 Sellers can download and print this label. When the QR code is scanned, the system shall automatically update the status to "Shipped" and move the payment process forward.
Rationale	<i>Managing shipping manually is one of the main reasons sellers leave a marketplace. Creating automatic labels saves time, prevents errors, and makes it easy for everyone to track the package within the app.</i>

Function ID	F-25
Function Name	Reverse Logistics (Returns) Workflow

MoSCoW Priority	MUST HAVE
User Requirement	The system must provide a clear way for buyers to return items and report problems if an item is not as described, within a set time limit.
System Requirements	25.1 The system must allow a buyer to send a Return Request within 48 hours after the item is Confirmed as Delivered. The request must include: the buyer's ID, the transaction ID, the reason (Not as Described / Damaged / Not Received), up to 5 photos as proof, and the time it was sent. 25.2 When a Return Request is sent, the payment (Escrow) will change from Delivered to Disputed. This stops the automatic payment timer and sends the case to the Problem Solving Center (F-29). 25.3 If an admin approves the return, the system will send the money back to the buyer (all or some, as decided by the admin), create a return shipping label, and list the item as Available for sale again once it is sent back.
Rationale	<i>Handling returns is important for trust. If there is no fair way to get money back for bad items, people will not use the platform, especially for expensive clothes.</i>

Function ID	F-26
Function Name	Bundle Discount Logic
MoSCoW Priority	COULD HAVE
User Requirement	The system could automatically apply a discount if a buyer buys more than one item from the same seller.
System Requirements	26.1 The system may calculate a "Bundle Saving" in the cart: for example, 5% off for 2 items, or 10% off for 3 items from the same shop. 26.2 This discount is intended to appear clearly in the cart and at checkout so the buyer knows exactly how much they are saving. 26.3 The system could combine this discount with other promotions, ensuring that the buyer gets the best deal while the seller still gets a fair price.
Rationale	<i>Bundle discounts encourage buyers to buy more items at once. This increases sales for the seller and reduces the total shipping effort, which is better for the environment.</i>

Function ID	F-27
Function Name	Payout Scheduling Engine
MoSCoW Priority	MUST HAVE
User Requirement	The system shall automatically plan and send payments to sellers for all finished sales based on a set schedule.

System Requirements	27.1 The system will run a Payment Task every day at 02:00 UTC. It will look for sales where the money is Released, the payment is Pending, and it is time to pay the seller. 27.2 For these sales, the system will: start a bank transfer to the seller's account, change the status to Processing, record the payment details (ID, amount, fees, and time), and send a message to the seller. 27.3 Admins can change how many days to wait before paying the seller (default: 2 business days) in the settings without needing to change the app's code.
Rationale	<i>Sellers will not use a platform if they don't know when they will get paid. Reliable payments help sellers stay on the platform and manage their money.</i>

Function ID	F-29
Function Name	Dispute Mediation Hub
MoSCoW Priority	MUST HAVE
User Requirement	The system shall give admins and moderators a special screen to look at proof and make a final decision on problems.
System Requirements	29.1 The system will create a Dispute record linked to a Return Request. It will show: who reported it, the seller's ID, proof (chat history, photos, item description), an admin ID, how urgent it is (P1 / P2 / P3), and the status (Open / Reviewing / Solved). 29.2 The admin screen will show a side-by-side view of the original item description, the condition report, the repair history, and the buyer's photos. This helps the admin make a fair decision based on the facts. 29.3 Once a decision is made, the admin chooses an outcome (Refund Buyer / Pay Seller / Partial Refund). The system will then move the money, update the item's status, and save a permanent record of the decision and the proof used.
Rationale	<i>Problems will happen in any shop. Without a clear way to fix them, arguments get worse and users lose trust. A formal process ensures things are fair and consistent.</i>

Function ID	F-30
Function Name	Collaborative Style Boards
MoSCoW Priority	COULD HAVE
User Requirement	The system could allow users to create public "Style Boards" and add items from different sellers to them.
System Requirements	30.1 The system may allow users to give their boards a title and description, and add up to 50 items. These boards can be public or archived. 30.2 These boards could be viewed by anyone, even without logging in. Every item on the board will show the seller's name and a link to the original item page.

	30.3 Users may follow a Style Board. To avoid too many alerts, the system will only notify followers if the creator adds 3 or more items in one day.
Rationale	<i>Style boards make browsing more fun and social. They help buyers discover new items and act as a free marketing tool for sellers when users share their boards.</i>

Function ID	F-31
Function Name	Live Drop Notifications
MoSCoW Priority	SHOULD HAVE
User Requirement	The system should notify all followers instantly when a seller posts a new item.
System Requirements	31.1 Users can "Follow" or "Unfollow" their favorite sellers. A seller's profile shall display the total number of followers and a simple button for users to follow them. 31.2 The moment a seller publishes a new item, the system shall automatically send an alert (Notification) to all their followers so they can see the new "Drop" immediately. 31.3 95% of notifications should arrive within 60 seconds of a listing being published.
Rationale	<i>Real-time notifications create excitement and help sellers sell their items faster. It also ensures that buyers don't miss out on unique, one-of-a-kind upcycled pieces.</i>

Function ID	F-32
Function Name	Seller Performance Analytics
MoSCoW Priority	COULD HAVE
User Requirement	The system could provide sellers with a private dashboard to track their sales and their positive impact on the environment.
System Requirements	32.1 The system could show the seller their total sales, how many people viewed their items, their average rating, and the "Eco-Credits" they earned. 32.2 The dashboard may also show "Sustainability Metrics," such as the total amount of CO2 and water saved through their sales, compared to other sellers. 32.3 This data is designed to be private (only the seller can see it) and will be updated regularly so the seller always has the latest information.
Rationale	<i>When sellers see their sales and environmental impact data, they can make better decisions about their prices and items. It also makes them feel proud of their contribution to the environment.</i>

Function ID	F-33
Function Name	Social Comment Threading

MoSCoW Priority	COULD HAVE
User Requirement	The system could allow users to write comments, reply to others, and use reactions (like emojis) on any public item page.
System Requirements	33.1 The system may support "Threaded Comments," meaning users can reply directly to a specific comment to keep the conversation organized. 33.2 To keep things readable, the system could limit replies to 3 levels deep. 33.3 Sellers is intended to have the power to "Pin" an important comment (like a helpful sizing tip) to the top of the conversation.
Rationale	<i>Comments and styling advice help buyers make decisions faster. It creates a community feeling and makes the platform more interactive than a regular shop.</i>

Function ID	F-34
Function Name	Report & Flagging Logic
MoSCoW Priority	MUST HAVE
User Requirement	The system shall allow users to report (flag) any post or person that breaks the rules, leading to an automatic response based on how many reports they get.
System Requirements	34.1 The system shall create a "Report Record" every time someone flags a post or a user. It will save who reported, who was reported, the reason (like fake items or bad behavior), and the date. The system shall block any user from reporting the same thing twice in 24 hours. 34.2 The system shall take automatic action in three stages: Stage 1 (1–2 reports): Send the post to the admin for a manual check. Stage 2 (3–5 reports): Apply a "Shadow-ban" (the post becomes hidden from everyone except the owner). Stage 3 (6+ reports): Fully suspend the user's account. 34.3 The system shall keep a permanent record (Audit Log) of every punishment stage. Only a "Lead Mediator" or "Super Admin" shall have the power to undo a shadow-ban or a suspension.
Rationale	<i>Fake items and bad behavior hurt the reputation of the platform. By having a system that automatically increases the punishment, we stop bad users quickly and keep the community safe and trustworthy.</i>

Function ID	F-35
Function Name	Mentor-Mentee Pairing
MoSCoW Priority	COULD HAVE
User Requirement	The system could connect expert upcyclers with beginners to help them learn how to repair and modify clothes.

System Requirements	35.1 Experienced users (with high trust scores and many sales) could volunteer as "Mentors" and list the skills they can teach. 35.2 The system may use a simple algorithm to match a beginner (Mentee) with the best available mentor based on the skills they want to learn. 35.3 Once matched, the system could open a private chat for them for 90 days so they can exchange advice and tips.
Rationale	<i>Helping people learn upcycling skills is key to growing our community. By connecting experts with beginners, we ensure that more people know how to save clothes from being wasted.</i>

Function ID	F-36
Function Name	Market Trend Reporting
MoSCoW Priority	SHOULD HAVE
User Requirement	The system should provide a public dashboard showing the most popular materials, brands, and categories on the platform.
System Requirements	36.1 The system shall update a "Top 10" list every day. This list will show which materials, brands, and categories were traded the most over the last 30 days. 36.2 This trend data shall be public and easy to load, ensuring that all users can see what is currently "trending" in the marketplace. 36.3 The trends dashboard shall show these items as ranked lists. Users can click on any trend (like "Cotton" or "Vintage") to instantly see all items in that category.
Rationale	<i>When sellers know what is in high demand, they can list items that are more likely to sell. This data helps the community grow and makes it easier for buyers to find popular styles.</i>

Function ID	F-37
Function Name	Dynamic Commission Modifier
MoSCoW Priority	SHOULD HAVE
User Requirement	The system should allow admins to set special, limited-time discounts on platform fees for specific categories.
System Requirements	37.1 Admins should be able to create "Fee Rules." For example, they can set a rule for "0% fees on all Upcycled items during Earth Day week." 37.2 When a user buys something, the system shall automatically check if there is a special fee rule for that category. If not, the standard fee will be applied. 37.3 Admins should be able to easily create, edit, or stop these special offers through their admin panel without needing to change the website's code.
Rationale	<i>Special promotions (like "Zero-Fee Fridays") are great for attracting new users and encouraging people to post more items. This tool gives the marketing team the power to start campaigns quickly.</i>

Function ID	F-39
Function Name	Role-Based Access Control (RBAC)
MoSCoW Priority	MUST HAVE
User Requirement	The system shall manage access by defining clear roles for each stakeholder, ensuring they only have the permissions needed for their specific tasks.

System Requirements	37.1 The system shall define the following access levels: Guest (can only browse and register), Registered User (full marketplace features), Admin/Moderator (can verify badges, resolve disputes, review flags), External System API (Payment Gateway and Shipping Provider for updating transaction statuses). 37.2 Access shall be verified at every login. The system shall check the user's role and only display the features and pages they are authorized to use. 37.3 Administrative actions, such as changing a user's role or banning an account, shall be restricted to the Admin role only and must be recorded in a "Security Audit Log."
Rationale	<i>By assigning permissions based on our specific stakeholders, we protect the system from unauthorized use. This ensures that a regular buyer cannot change a price or access another user's payment data, which is critical for security and trust.</i>

3.b Requirements Priorities — MoSCoW Scheme

The following table summarises the MoSCoW classification of all 42 functions. Functions 14, 23, 28, 38, 40, 41, and 42 are classified as Won't Have for this version.

Priority	Count	Function IDs	Rationale for Priority Level
MUST HAVE	16	F-02, F-03, F-08, F-10, F-11, F-12, F-15, F-16, F-19, F-20, F-22, F-25, F-27, F-29, F-34, F-39	Essential for core platform operation, legal compliance, or user safety. Platform cannot launch without these.
SHOULD HAVE	10	F-01, F-04, F-06, F-09, F-17, F-18, F-24, F-31, F-36, F-37	High value; significant negative impact if omitted. Should be included if resources allow.
COULD HAVE	9	F-05, F-07, F-13, F-21, F-26, F-30, F-32, F-33, F-35	Desirable but not critical. Can be deferred to a subsequent release without significant impact.
WON'T HAVE	7	F-14, F-23, F-28, F-38, F-40, F-41, F-42	Explicitly excluded from this version. Candidate for future roadmap.

3. Non-Functional Requirements

3.a Categories of Non-Functional Requirements

ReThread adopts the following NFR categories, aligned with the ISO/IEC 25010 quality model and Sommerville's classification:

- Performance — Response time, throughput, and resource utilization under defined load conditions.
- Reliability — System availability, fault tolerance, and data integrity under failure conditions.
- Security — Data protection, authentication, authorization, and privacy compliance.
- Usability — Ease of learning, efficiency of use, and user error prevention.
- Scalability — Ability to handle growth in users, listings, and transactions without architectural change.
- Maintainability — Code quality, test coverage, and ease of modification and extension.
- Sustainability — Environmental responsibility of the platform's own infrastructure.

3.b NFR Specification (with Fit Criteria)

NFR Name	Category	Requirement	Fit Criteria (Testable)
Marketplace Feed Load Time	Performance	The marketplace feed shall load within 3 seconds for 95% of requests under 1,000 concurrent users.	Load test with JMeter: 1,000 concurrent users, 95th percentile response time $\leq$ 3 seconds.
Carbon Calculator Latency	Performance	The Carbon Footprint Calculator shall return results within 500ms.	Unit benchmark: 1,000 consecutive calculations complete in < 500 seconds total.
System Availability (Uptime)	Reliability	The system shall achieve 99.5% uptime, measured monthly.	Monthly uptime monitoring report: downtime < 3.6 hours/month.
Transaction Data Integrity	Reliability	The Escrow Payment Simulation shall be ACID-compliant with zero data loss on system failure.	Chaos testing: simulate DB crash mid-transaction; verify escrow state is recoverable to last committed state.
Password Hashing Security	Security	All user passwords shall be hashed using bcrypt with a cost factor of $\geq$ 12.	Code review + automated static analysis confirms no plain-text password storage.
API Authentication Security	Security	All API endpoints shall require JWT authentication except public browse and search.	Penetration test: unauthorized requests to protected endpoints return HTTP 401.
Location Data Privacy	Security	User geolocation data (Swap Meet Finder) shall be opt-in and encrypted at rest.	Data audit: geolocation column uses AES-256 encryption; users without consent have NULL value.
Listing Process Efficiency	Usability	A new user shall be able to create and publish their first listing within 10 minutes without assistance.	Usability test with 5 new users: median time-to-first-listing $\leq$ 10 minutes.
Assessment Flow Simplicity	Usability	The Condition Assessment Assistant shall complete in $\leq$ 7 steps for any garment category.	UX walkthrough: no garment path through the decision tree exceeds 7 screens.

Horizontal Scalability	Scalability	The system architecture shall support horizontal scaling to handle 10× peak load with no code changes.	Architecture review: stateless services confirmed; session state stored in Redis; DB supports read replicas.
Automated Test Coverage	Maintainability	The codebase shall maintain a test coverage of ≥ 80% for all business logic modules.	CI pipeline coverage report: coverage badge shows ≥ 80% on main branch.
Eco-Friendly Infrastructure	Sustainability	The system's own server infrastructure shall use a cloud provider with a renewable energy commitment.	Vendor documentation confirms ≥ 90% renewable energy usage for selected data center regions.

3.c Impact of NFRs on System Architecture

The non-functional requirements above directly shape the following architectural decisions:

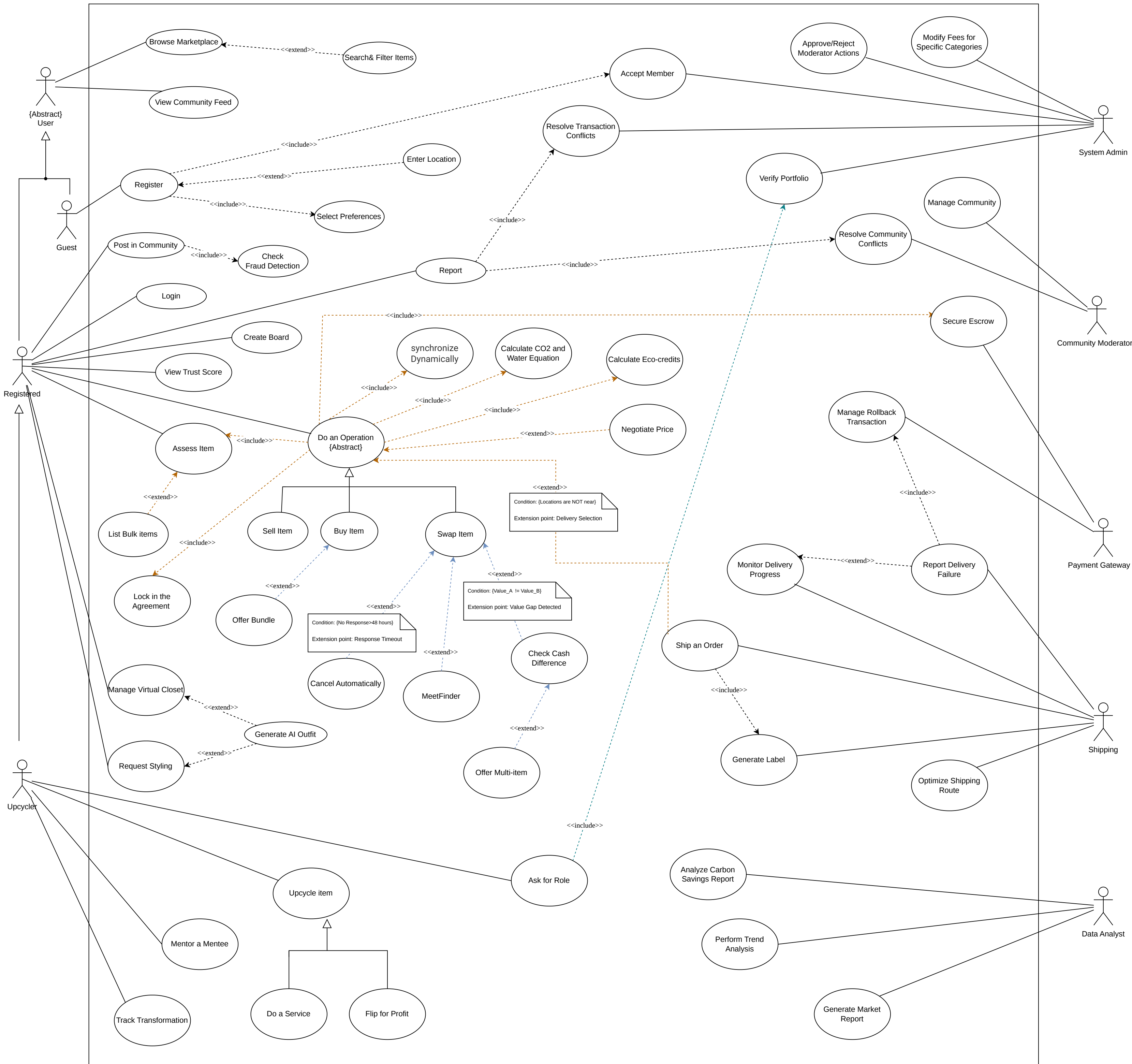
- **Performance (NFR-01, NFR-02):** The system shall use a CDN for static assets, Redis caching for the Style Feed and Trends API, and asynchronous processing (message queue) for notification dispatch — decoupling slow operations from the critical request path.
- **Reliability (NFR-03, NFR-04):** The system shall use a relational database (PostgreSQL) with ACID guarantees for all financial and swap-agreement transactions. A multi-availability-zone deployment shall eliminate single points of failure.
- **Security (NFR-05, NFR-06, NFR-07):** The system shall adopt a layered security model: JWT at the API gateway, RBAC at the service layer, and column-level encryption for sensitive fields. A secrets manager (e.g., HashiCorp Vault) shall manage credentials.
- **Scalability (NFR-10):** All application services shall be stateless (session state in Redis), enabling horizontal auto-scaling. The database shall use read replicas for query-heavy operations (feed, search, analytics).
- **Maintainability (NFR-11):** The system shall be built with a modular microservices-inspired architecture (or well-separated domain modules in a monolith), with a CI/CD pipeline enforcing code coverage gates before deployment.

5. References

- Ellen MacArthur Foundation. (2017). A New Textiles Economy: Redesigning Fashion's Future. ellenmacarthurfoundation.org.
- ThredUp. (2023). 2023 Resale Report. thredup.com/resale-report.
- Fletcher, K. (2014). Sustainable Fashion and Textiles: Design Journeys (2nd ed.). Routledge.
- WCAG Working Group. (2018). Web Content Accessibility Guidelines (WCAG) 2.1. W3C Recommendation.
- Egyptian Consumer Protection Authority. Law No. 181 of 2018 on Consumer Protection.
- OWASP Foundation. (2021). OWASP Top Ten. owasp.org.
- Sommerville's classification

Appendix A: Glossary

Term	Definition
Eco-Credits	A virtual currency awarded to users for environmentally beneficial transactions. Not redeemable for real money.
Trust Score	A dynamically computed reliability metric for sellers and swappers, based on description accuracy, shipping speed, and swap completion rate.
Escrow	A simulated financial arrangement where a third party (the platform) holds funds until both parties fulfil their transaction obligations.
Transformation Log	A structured record linking Before and After images and a description of modifications to a garment, required for Upcycle listings.
RBAC	Role-Based Access Control — a method of regulating access to system resources based on the roles of individual users.
Atomic Transaction	A database operation that is treated as a single indivisible unit: either all changes succeed or none are applied.
Shadow-Ban	A moderation action that hides a user's content from other users without notifying the banned user.
Digital Closet	A private user-managed list of owned items not publicly listed on the marketplace.
Volere Shell	A structured requirements template by Robertson & Robertson that includes a Goal and a Fit Criterion for each non-functional requirement.
JBA Framework	John Boardman Associates stakeholder identification methodology using five lenses: system goals, affected parties, decision-makers, subject matter experts, and regulators.
MoSCoW	A requirements prioritisation scheme: Must Have, Should Have, Could Have, Won't Have.



Describing Use Cases [[Browse Marketplace](#)]

ID Number – Name:	UC-1 – Browse Marketplace
Short-Description / Goal:	A user (Guest, Registered User, or Upcycler) browses all available item listings on the marketplace and applies filters, sorting, and search to find relevant items.
Preconditions:	System is online and item listings are available.
Postconditions:	User views a filtered/sorted list of items matching their criteria.
Error Situations – System State in the Event of an Error:	No items match the applied filters – System displays an empty results message with suggestions to broaden the search.
Actor(s) / Initiator(s):	Guest, Buyer, Seller, Swapper, Upcycler
Trigger:	User opens the marketplace or enters a search query.
Standard Process (Main Success Scenario):	(1) User navigates to the Marketplace page. (2) System displays all active listings by default. (3) User applies filters (e.g., category, material, condition, price range, upcycled-only). (4) User applies sorting (e.g., newest, price low-to-high, trust score). (5) System updates and displays the filtered and sorted results. (6) User clicks on an item to view its full details.
Alternative Processes (Alternative/Other Scenarios):	(3') User types a keyword in the search bar – System returns matching results. (5') No results found – System suggests broadening filters or browsing trending items.

Describing Use Cases [[Register](#)]

ID Number – Name:	UC-2 – Register
Short-Description / Goal:	A new visitor creates an account on the platform by providing their information and completing a preference selection to personalize their experience.
Preconditions:	User does not have an existing account. User has a valid email address and password.
Postconditions:	A new account is created and stored. User's preferences are saved and a personalized feed is generated. User is redirected to the home feed.
Error Situations – System State in the Event of an Error:	Email is already registered – System prompts user to log in or reset password. Invalid data format – System highlights the invalid fields without clearing the form.
Actor(s) / Initiator(s):	Guest
Trigger:	Guest clicks the 'Register' button.
Standard Process (Main Success Scenario):	(1) Guest clicks 'Register'. (2) System displays the registration form (name, email, password, location). (3) Guest fills in the required information. (4) System validates all fields and checks email uniqueness. (5) System stores the new account data. (6) System launches the Preference Selection wizard (style aesthetics, sizes, favourite materials, upcycle interest level) — <i>similar to Pinterest onboarding</i>. (7) User completes preference selection. (8) System curates a personalised home feed based on selected preferences. (9) User is redirected to their personalised home feed.
Alternative Processes (Alternative/Other Scenarios):	(4') Email already exists – System notifies user and offers a 'Log In' link. (7') User skips preference selection – System shows a default trending feed and prompts preferences later.

Describing Use Cases [[Login](#)]

ID Number – Name:	UC-3 – Login
Short-Description / Goal:	A registered user authenticates into the platform using their email and password to access their account and perform actions.
Preconditions:	User has a registered account. System is online.
Postconditions:	User is authenticated and redirected to their personalised home feed. Session is created.
Error Situations – System State in the Event of an Error:	Invalid credentials – System displays 'Login Failed' message; account is not locked after first attempt.
Actor(s) / Initiator(s):	Registered User (Buyer, Seller, or Upcycler — role is determined by actions taken, not fixed to the account)
Trigger:	User clicks the 'Login' button on the landing page or is redirected after attempting a restricted action.
Standard Process (Main Success Scenario):	(1) User clicks 'Login'. (2) System displays the login form (email, password). (3) User enters their credentials. (4) System validates credentials against the database. (5) Credentials are correct – System creates a session. (6) System redirects user to their personalised home feed.
Alternative Processes (Alternative/Other Scenarios):	(1') User has no account – System offers a 'Register' link. (4') Credentials are invalid – System displays 'Login Failed' and allows retry.

Describing Use Cases [[Ask for Upcycling Role](#)]

ID Number – Name:	UC-4 – Ask for Upcycling Role
Short-Description / Goal:	A registered user applies for the Upcycler role by submitting a portfolio for admin review, allowing them to list upcycled items and access creator-specific features.
Preconditions:	User is logged in as a standard Registered User. User has a portfolio of upcycling work ready to upload.
Postconditions:	[Approved] User's role is updated to Upcycler; user is notified and gains access to Upcycler features. [Rejected] User is notified with a rejection reason and remains a standard Registered User.
Error Situations – System State in the Event of an Error:	Portfolio upload fails – System displays an upload error and allows retry without losing other form data.
Actor(s) / Initiator(s):	Registered User, System Admin
Trigger:	Registered User clicks 'Apply for Upcycler Role'.
Standard Process (Main Success Scenario):	(1) User navigates to their profile and clicks 'Apply for Upcycler Role'. (2) System displays the application form. (3) User uploads portfolio images/links and submits the application. (4) System notifies the Admin of a pending application. (5) Admin reviews the portfolio. (6) Admin approves the application. (7) System updates the user's role to Upcycler. (8) System sends an approval notification to the user. (9) User can now access Upcycler-specific features (Track Transformation, Upcycle Item, etc.).
Alternative Processes (Alternative/Other Scenarios):	(6') Admin rejects the application – Admin enters a rejection reason. (7') System sends a rejection notification with the reason to the user. (8') User remains a standard Registered User and may reapply.

Describing Use Cases [[Post in Community](#)]

ID Number – Name:	UC-5 – Post in Community
Short-Description / Goal:	A registered user creates a post in the community feed. Post types include 'Style Request' (styling advice on an item) and 'Inspire Me' (finding a similar item). All posts pass through automated fraud detection before going live.
Preconditions:	User is logged in. Community feed is accessible.
Postconditions:	[Accepted] Post is published in the community feed and visible to other users. [Rejected] Post is blocked and user is notified of the violation.
Error Situations – System State in the Event of an Error:	Post content triggers fraud detection – System auto-rejects the post and notifies the user with a reason.
Actor(s) / Initiator(s):	Buyer, Seller, Swapper, Upcycler, Moderator
Trigger:	User clicks 'New Post' in the community feed.
Standard Process (Main Success Scenario):	(1) User clicks 'New Post'.as 'Style Request' or 'Inspire Me'. (1.1) [Style Request] User uploads a photo of an item and asks how to style it. (1.2) [Inspire Me] User uploads a photo and asks where to find a similar item; system checks marketplace first, then post to the community. (2) User submits the post. (3) System runs automated fraud detection on the content (prohibited words, misleading claims). (4) Post passes detection – System publishes the post. (5) Other users can comment with suggestions or recommendations. (6) Post author selects a preferred response; system may reward the idea's owner with Eco-Credits.
Alternative Processes (Alternative/Other Scenarios):	(3') Fraud detection flags the post – System rejects it and notifies the user.

Describing Use Cases [Manage Community]

ID Number – Name:	UC-6 – Manage Community
Short-Description / Goal:	A Community Moderator oversees the community feed by approving or rejecting posts, handling harassment reports, and resolving community conflicts. Major decisions require System Admin approval.
Preconditions:	Moderator is logged in with Moderator-level permissions. Community content or reports are pending review.
Postconditions:	Flagged content is actioned (approved/rejected/removed). Community conflict is resolved or escalated to Admin.
Error Situations – System State in the Event of an Error:	Moderator attempts an action that requires Admin authorization – System blocks the action and sends an escalation request to Admin.
Actor(s) / Initiator(s):	Community Moderator, System Admin
Trigger:	A post is flagged by a user, fraud detection, or appears in the moderation queue.
Standard Process (Main Success Scenario):	(1) Moderator opens the moderation dashboard. (2) System displays a queue of flagged posts/comments/accounts. (3) Moderator reviews the flagged content. (4) Moderator decides to approve or reject the content. (5) For significant actions (e.g., banning an account), Moderator submits a decision for Admin approval. (6) Admin reviews and approves or overrides the Moderator's decision. (7) System applies the final action and notifies the relevant users.
Alternative Processes (Alternative/Other Scenarios):	(4') Harassment or bullying is confirmed – Moderator escalates directly to Admin for account suspension. (6') Admin overrides Moderator's decision – System logs the override and applies Admin's ruling.

Describing Use Cases [[Mentor a Mentee](#)]

ID Number – Name:	UC-7 – Mentor a Mentee
Short-Description / Goal:	An experienced Upcycler (Mentor) accepts a mentorship request from a novice Upcycler (Mentee), supervises their upcycling project, and earns a badge or Eco-Credit reward upon the Mentee's successful sale.
Preconditions:	Mentor is a verified Upcycler. Mentee is a verified Upcycler working on an upcycling project. Mentee has submitted a mentorship request.
Postconditions:	Mentee receives guidance on their upcycling project. [Upon successful sale] Mentor receives Eco-Credits. System updates both Mentor and Mentee Badge rankings.
Error Situations – System State in the Event of an Error:	Mentor becomes unresponsive – System reassigns the Mentee to another available Mentor after a defined timeout.
Actor(s) / Initiator(s):	Upcycler (Mentor), Upcycler (Mentee)
Trigger:	Mentee clicks 'Request a Mentor' during an upcycling project.
Standard Process (Main Success Scenario):	(1) Mentee initiates an upcycling project and selects 'I need a mentor'. (2) System matches the Mentee with available, high-ranked Mentors. (3) Mentor receives a mentorship request and accepts. (4) Mentor supervises the Mentee's transformation process (reviews before/after photos, gives advice). (5) Mentee completes the project and lists the item. (6) Item is sold successfully. (7) System rewards the Mentor with Eco-Credits. (8) System updates both users' Badge rankings.
Alternative Processes (Alternative/Other Scenarios):	(3') Mentor declines the request – System offers the request to the next available Mentor. (1') Mentee selects 'I did it on my own' – Mentorship flow is skipped; Mentee documents independently.

Describing Use Cases [[Report](#)]

ID Number – Name:	UC-8 – Report
Short-Description / Goal:	A user reports a post, comment, or account for a violation. Financial disputes are escalated to Admin; community-related reports are handled by the Moderator.
Preconditions:	User is logged in. The reported content or account exists in the system.
Postconditions:	Report is logged and routed to the appropriate handler (Admin or Moderator). Appropriate action is taken based on the report type.
Error Situations – System State in the Event of an Error:	Report submission fails due to a system error – System displays a retry option and logs the failure.
Actor(s) / Initiator(s):	Registered User, Upcycler, Community Moderator, System Admin
Trigger:	User clicks 'Report' on a post, comment, or user profile.
Standard Process (Main Success Scenario):	(1) User clicks 'Report' on the target content or account. (2) System displays a report form with reason categories (e.g., Fraud, Harassment, Counterfeit, Misleading Listing). (3) User selects a reason and optionally adds a description. (4) User submits the report. (5) System logs the report and routes it: financial disputes → Admin; community issues → Moderator. (6) Handler reviews and takes action (warn, remove content, suspend account). (7) System notifies the reporting user of the outcome.
Alternative Processes (Alternative/Other Scenarios):	(6') Content is found non-violating – Report is dismissed; reporting user is notified.

Describing Use Cases [[Assess Item](#)]

ID Number – Name:	UC-9 – Assess Item
Short-Description / Goal:	A registered user (Seller or Swapper_A or Upcycler) creates a detailed item listing by providing photos, condition tags, material type, and description. Supports single-item and bulk listing.
Preconditions:	User is logged in as a Registered User or Upcycler. User has an item to list (New or from Virtual Closet).
Postconditions:	Item is listed on the marketplace and visible to other users. Item is available for purchase, swapping, or offers.
Error Situations – System State in the Event of an Error:	Fraud detection flags a misleading description – Listing is rejected; user is notified with a reason. Required fields are missing – System highlights incomplete fields and prevents submission.
Actor(s) / Initiator(s):	Seller, Swapper_A, Upcycler
Trigger:	User clicks 'List an Item' or 'Quick List' from their Virtual Closet.
Standard Process (Main Success Scenario):	(1) User clicks 'List an Item'. (2) System presents a guided listing form. (3) User uploads photos (min. 1 required). (4) User fills in item details: title, category, material type, condition tags (e.g., 'Pilling on sleeves'), size, price. (5) User sets availability: For Sale, For Swap, or Both. (6) User submits the listing. (7) System runs fraud/content moderation check. (8) Listing passes check – System publishes the item to the marketplace.
Alternative Processes (Alternative/Other Scenarios):	(1') User selects 'Bulk Listing' – System allows uploading multiple items at the same time. (7') Fraud check fails – System rejects listing and notifies user.

Describing Use Cases [[Manage Virtual Closet](#)]

ID Number – Name:	UC-10 – Manage Virtual Closet
Short-Description / Goal:	A registered user manages a private digital inventory of clothing they own but have not listed for sale or swap. Items in the Virtual Closet can be used for AI outfit generation or quickly converted to a marketplace listing.
Preconditions:	User is logged in. User has items they wish to add to their private closet.
Postconditions:	Item is saved in the Virtual Closet and available for AI outfit suggestions. [If listed] Item transitions from Virtual Closet to active marketplace listing.
Error Situations – System State in the Event of an Error:	Image upload fails – System prompts user to retry without losing other entered data.
Actor(s) / Initiator(s):	Buyer, Seller, Swapper, Upcycler
Trigger:	User navigates to 'My Closet' from their profile.
Standard Process (Main Success Scenario):	(1) User opens 'My Closet'. (2) System displays all items currently in the Virtual Closet. (3) User clicks 'Add Item' and uploads a photo with basic details (name, material, colour). (4) System saves the item to the closet. (5) Closet items are available as input for the Generate AI Outfit feature.
Alternative Processes (Alternative/Other Scenarios):	(3') User selects an existing closet item and clicks 'List for Sale/Swap' – System pre-fills the Assess Item form with the closet item's details for quick listing. (3'') User removes an item from the closet – System deletes it from the private inventory.

Describing Use Cases [[Generate AI Outfit](#)]

ID Number – Name:	UC-11 – Generate AI Outfit
Short-Description / Goal:	A registered user submits an item (from their Virtual Closet or the marketplace) to an AI assistant that suggests a complete outfit and provides an AR fitting room to visualize the result, reducing returns.
Preconditions:	User is logged in. User has at least one item in their Virtual Closet or has selected a marketplace item.
Postconditions:	AI-generated outfit suggestion is displayed. User can view himself (optionally) in the AR fitting room. User can add suggested items directly to cart.
Error Situations – System State in the Event of an Error:	AI service is unavailable – System displays an error message and offers to post a Style Request to the community instead.
Actor(s) / Initiator(s):	Registered User, Upcycler
Trigger:	User clicks 'Generate Outfit' on an item in their closet or on a marketplace listing.
Standard Process (Main Success Scenario):	(1) User selects an item and clicks 'Generate Outfit'. (2) System sends the item and user's preference profile to the AI engine. (3) AI suggests a complete outfit by combining closet items and/or marketplace listings. (4) User views the outfit suggestion. (5) User tries the outfit in the AR Fitting Room. (6) User adds preferred items to cart or saves the outfit.
Alternative Processes (Alternative/Other Scenarios):	(3') AI cannot find enough matching items – System suggests posting a 'Style Request' to the community. (5') User skips AR and proceeds directly to adding items to cart.

Describing Use Cases [[Track Transformation](#)]

ID Number – Name:	UC-12 – Track Transformation
Short-Description / Goal:	An Upcycler documents the transformation of a garment by uploading before/after photos, describing modifications made, logging additional materials used, and sharing an Upcycling Story to build credibility and trust.
Preconditions:	User is logged in as an Upcycler. User has an item undergoing or completed an upcycling transformation.
Postconditions:	Transformation log is saved and attached to the item listing. Carbon savings and Eco-Credits are calculated and displayed. Upcycling Story is visible on the item listing page.
Error Situations – System State in the Event of an Error:	Photo upload fails – System allows partial save and prompts user to complete the log before publishing.
Actor(s) / Initiator(s):	Upcycler
Trigger:	Upcycler starts a new upcycling project or opens an existing draft transformation log.
Standard Process (Main Success Scenario):	(1) Upcycler creates a new transformation log. (2) Upcycler uploads 'Before' photo(s) of the original garment. (3) Upcycler describes the modifications made (e.g., 'Removed sleeves, added embroidery, re-dyed with natural dye'). (4) Upcycler logs additional materials used (type, quantity). (5) Upcycler uploads 'After' photo(s) of the finished item. (6) Upcycler writes the Upcycling Story. (7) System calculates carbon savings based on material type and weight. (8) System awards Eco-Credits based on impact score. (9) Transformation log is saved and linked to the item listing.
Alternative Processes (Alternative/Other Scenarios):	(4') Upcycler indicates zero additional materials used – System records it as a repair-only transformation. (6') Upcycler skips the story – System saves the log without a story; story can be added later.

Describing Use Cases [[Upcycle Item](#)]

ID Number – Name:	UC-13 – Upcycle Item
Short-Description / Goal:	An Upcycler selects an upcycling path: (A) transform an owned item to resell for profit (Flip for Profit), or (B) perform an upcycling service on a client's item and return it. Path B requires a Security Bond to protect the client.
Preconditions:	User is logged in as a verified Upcycler. Item to be upcycled is available (owned by Upcycler or submitted by a client).
Postconditions:	[Path A] Upcycled item is listed for sale on the marketplace with a transformation log. [Path B] Upcycled item is returned to the client; Security Bond is released.
Error Situations – System State in the Event of an Error:	[Path B] Item is damaged during the service – Security Bond is used to compensate the client; trust score is reduced.
Actor(s) / Initiator(s):	Upcycler, Registered User (as client in Path B)
Trigger:	Upcycler clicks 'Start Upcycling Project'.
Standard Process (Main Success Scenario):	(1) Upcycler clicks 'Start Upcycling Project'. (2) System asks: 'Do you already own the piece?' or 'Are you providing a service?' (3a) [Path A – Flip for Profit] Upcycler selects their item (owned or to be purchased from the site). (4a) Upcycler performs the upcycling work and logs the transformation (see UC-12). (5a) Upcycler lists the finished item for sale on the marketplace. (3b) [Path B – Service] Client submits the item and pays a Security Bond (equivalent to item's estimated value). (4b) Upcycler performs the service and logs the transformation. (5b) Upcycler returns the item to the client. (6b) Client confirms receipt – Security Bond is released to the Upcycler.
Alternative Processes (Alternative/Other Scenarios):	(3a') Upcycler buys an item from the marketplace to upcycle – Standard Buy Item flow is triggered first. (5b') Client disputes the work quality – Admin reviews and decides on Bond release or partial refund.

Describing Use Cases [[Sell Item](#)]

ID Number – Name:	UC-14 – Sell Item
Short-Description / Goal:	A registered user lists an item for sale, sets a price with an optional negotiation floor, and completes the sale through the platform's secure transaction flow.
Preconditions:	User is logged in. Item is assessed and listed (see UC-9). Item is marked as 'For Sale'.
Postconditions:	Item is sold and marked as unavailable. Funds are transferred to escrow pending delivery confirmation. Dynamic Sync removes item from all active listings.
Error Situations – System State in the Event of an Error:	Item is simultaneously engaged in a swap – System locks the item and rejects the sale attempt until the swap is resolved.
Actor(s) / Initiator(s):	Registered User (Seller), Registered User (Buyer)
Trigger:	A buyer initiates a purchase or a seller receives an offer on their listed item.
Standard Process (Main Success Scenario):	(1) Seller lists the item with a set price and configures a negotiation floor (minimum acceptable price). (2) Buyer views the listing and clicks 'Buy Now' or submits a price offer. (3) [If offer] System checks if the offer meets the Seller's auto-accept threshold. (4) Offer is accepted (manually or automatically) – Sale proceeds. (5) System triggers Agreement Lock-in: item description and price are frozen. (6) Buyer completes payment – Funds are held in Escrow. (7) Seller ships the item (see UC-19). (8) Buyer confirms receipt – Funds are released to Seller. (9) System updates Seller's trust score and Eco-Credits.
Alternative Processes (Alternative/Other Scenarios):	(2') Multiple items from same seller – System applies bundle discount automatically. (3') Offer is below auto-decline threshold – System automatically rejects the offer.

Describing Use Cases [[Buy Item](#)]

ID Number – Name:	UC-15 – Buy Item
Short-Description / Goal:	A registered user purchases an item from the marketplace, with an optional price negotiation phase, bundle discount for multiple items from the same seller, and secure payment via Escrow.
Preconditions:	User is logged in. Item is listed as 'For Sale' and not locked in another transaction. Buyer has a valid payment method.
Postconditions:	Purchase is confirmed. Item is locked and removed from the marketplace. Funds are held in Escrow until delivery is confirmed.
Error Situations – System State in the Event of an Error:	Payment fails – System notifies the buyer, order is cancelled, and item is re-listed.
Actor(s) / Initiator(s):	Registered User (Buyer), Registered User (Seller), Payment Gateway
Trigger:	Buyer clicks 'Buy Now' or 'Make an Offer' on an item listing.
Standard Process (Main Success Scenario):	(1) Buyer clicks 'Buy Now' on a listing. (2) System displays the order summary (item price + shipping cost + platform fee). (3) Buyer selects a payment method and enters payment details. (4) System validates payment details. (5) Payment is confirmed – Funds are transferred to Escrow. (6) System notifies the Seller to prepare and ship the item. (7) Seller ships the item. (8) Buyer confirms receipt. (9) Escrow releases funds to Seller.
Alternative Processes (Alternative/Other Scenarios):	(1') Buyer clicks 'Make an Offer' – Negotiation flow is triggered (see UC-13 Negotiate Price). (2') Buyer has multiple items from same seller – System auto-applies bundle discount to the order total. (4') Payment details invalid – System highlights errors and prompts correction.

Describing Use Cases [[Swap Item](#)]

ID Number – Name:	UC-16 – Swap Item
Short-Description / Goal:	Two registered users propose and agree to exchange items. A value balancer handles price differences, a 48-hour auto-cancel timer prevents stale offers, and an escrow secures any cash top-up.
Preconditions:	Both users are logged in. Both items are listed as 'For Swap' and not locked in another transaction.
Postconditions:	Swap is agreed and locked in. Both items are removed from the marketplace. Shipping is arranged for both parties. [If cash top-up] Difference is held in Escrow and transferred upon delivery confirmation.
Error Situations – System State in the Event of an Error:	No response within 48 hours – System auto-cancels the offer and unlocks both items.
Actor(s) / Initiator(s):	Registered User (Swapper A), Registered User (Swapper B), Moderator, Shipping
Trigger:	Swapper A clicks 'Propose Swap' on Swapper B's listing.
Standard Process (Main Success Scenario):	(1) Swapper A selects their item(s) to offer in exchange for Swapper B's item. (2) System calculates the value difference between items. (3) If value gap exists – System notifies both parties and suggests a cash top-up or an additional item. (4) Swap offer is sent to Swapper B. (5) Swapper B accepts – Moderator reviews and approves the swap. (6) Agreement Lock-in is triggered: descriptions and terms are frozen. (7) Dynamic Sync removes both items from the marketplace. (8) Both parties arrange shipping. (9) Both confirm receipt – Swap is marked as complete; Eco-Credits and trust scores are updated.
Alternative Processes (Alternative/Other Scenarios):	(3') Swapper A adds an extra item to bridge the value gap – Multi-item offer flow is triggered. (5') Swapper B rejects – System notifies Swapper A and unlocks Swapper A's items.

Describing Use Cases [[Calculate Carbon Savings](#)]

ID Number – Name:	UC-17 – Calculate Carbon Savings
Short-Description / Goal:	The system automatically calculates the estimated CO ₂ savings for each completed transaction based on item material type and weight, displaying the result to users and contributing to platform-wide sustainability reports.
Preconditions:	A transaction (sale, swap, or upcycle) has been completed. Item material type and weight data are available in the listing.
Postconditions:	Carbon savings is calculated and attached to the transaction record. User's cumulative sustainability impact is updated on their profile. Data is available for platform-wide reports by the Data Analyst.
Error Situations – System State in the Event of an Error:	Material or weight data is missing – System uses category-level averages and flags the estimate as approximate.
Actor(s) / Initiator(s):	System (automated), Data Analyst
Trigger:	A transaction is marked as completed.
Standard Process (Main Success Scenario):	(1) Transaction is marked as complete. (2) System retrieves item material type and weight from the listing. (3) System applies the carbon and water savings algorithm (based on material type and avoided new production). (4) Calculated savings are stored in the transaction record. (5) Savings are displayed on the user's profile impact dashboard. (6) Eco-Credits are calculated and awarded based on the impact score. (7) Data is aggregated for the Data Analyst's market and sustainability reports.
Alternative Processes (Alternative/Other Scenarios):	(2') Material data is missing – System uses average values for the item's category and labels the result as an estimate.

Describing Use Cases [Secure Escrow]

ID Number – Name:	UC-18 – Secure Escrow
Short-Description / Goal:	The system holds a buyer's payment in a secure virtual vault after purchase confirmation and releases it to the seller only upon verified delivery. In case of non-delivery or dispute, funds are returned to the buyer.
Preconditions:	Buyer has confirmed and paid for an order. Payment Gateway has processed the transaction successfully.
Postconditions:	[Delivered & confirmed] Funds are released to Seller. [Dispute] Admin determines responsibility; funds are refunded or partially released accordingly.
Error Situations – System State in the Event of an Error:	Escrow system fails to hold funds – Transaction is cancelled and buyer is fully refunded immediately.
Actor(s) / Initiator(s):	Payment Gateway, System Admin, Registered User (Buyer), Registered User (Seller)
Trigger:	Buyer's payment is successfully processed by the Payment Gateway.
Standard Process (Main Success Scenario):	(1) Buyer completes payment. (2) Payment Gateway transfers funds to the Escrow vault. (3) System notifies Seller that payment is secured and order should be shipped. (4) Seller ships the item. (5) Buyer confirms receipt of the item. (6) System releases Escrow funds to the Seller. (7) Transaction is marked as complete.
Alternative Processes (Alternative/Other Scenarios):	(5') Buyer does not confirm receipt within the defined window – System escalates to Admin. (5'') Buyer reports non-delivery – Admin investigates; if confirmed, funds are refunded to Buyer and Seller's trust score is reduced. (5''') Item is returned – Rollback transaction is triggered and Escrow is reversed.

Describing Use Cases [[Ship an Order](#)]

ID Number – Name:	UC-19 – Ship an Order
Short-Description / Goal:	The Shipping actor processes an order by generating a shipping label with full item and recipient details, optimizing the delivery route to minimize carbon emissions, and updating the order status throughout transit until delivery is confirmed.
Preconditions:	Order is confirmed and payment is secured in Escrow. Seller has prepared the package. Buyer's delivery address is provided.
Postconditions:	Item is delivered to the buyer. Order status is updated to 'Delivered'. Escrow release is triggered.
Error Situations – System State in the Event of an Error:	Delivery route cannot be optimised – System uses the default route and logs the failure for review.
Actor(s) / Initiator(s):	Shipping, Seller, Buyer
Trigger:	Seller marks the order as 'Ready to Ship'.
Standard Process (Main Success Scenario):	(1) Seller marks the order as 'Ready to Ship'. (2) System generates a shipping label containing: buyer name, delivery address, item brief description, item ID, and order value. (3) System runs route optimization to select the most carbon-efficient delivery path. (4) Shipping actor picks up the package. (5) System sets order status to 'In Transit'. (6) Buyer can track the shipment in real time. (7) Package is delivered – Buyer confirms receipt. (8) System updates status to 'Delivered' and triggers Escrow release.
Alternative Processes (Alternative/Other Scenarios):	(4') [Swap Meet] Buyer and Seller are located nearby – System suggests a local meetup instead of shipping to eliminate emissions. (7') Buyer is unavailable for delivery – Shipping actor leaves a notice; status set to 'Delivery Attempted'.

Describing Use Cases [[Report Delivery Failure](#)]

ID Number – Name:	UC-20 – Report Delivery Failure
Short-Description / Goal:	A buyer reports that an order was not received or was received in an unacceptable condition. Admin investigates and determines responsibility, applying financial consequences to the responsible party.
Preconditions:	Order status is 'In Transit' or 'Delivered'. Buyer has not confirmed receipt within the expected window, or has flagged an issue.
Postconditions:	[Buyer at fault] Shipping cost is deducted from buyer; item is returned. [Seller at fault] Seller pays shipping cost; buyer is fully refunded; Seller's trust score is reduced.
Error Situations – System State in the Event of an Error:	Admin cannot determine responsibility – Funds are held in Escrow pending further evidence submission.
Actor(s) / Initiator(s):	Registered User (Buyer), System Admin, Shipping, Payment Gateway
Trigger:	Buyer clicks 'Report Delivery Issue' or delivery window expires without buyer confirmation.
Standard Process (Main Success Scenario):	(1) Buyer clicks 'Report Delivery Issue' and describes the problem. (2) System logs the report and notifies Admin. (3) Admin reviews shipping evidence (tracking history, seller's proof of dispatch). (4) Admin determines responsibility. (5a) [Seller responsible] System deducts shipping cost from Seller, refunds Buyer, and reduces Seller's trust score. (5b) [Buyer responsible] System deducts shipping cost from Buyer and arranges item return. (6) Escrow is settled according to Admin's decision.
Alternative Processes (Alternative/Other Scenarios):	(1') Delivery window expires with no buyer action – System auto-escalates to Admin after a defined period. (4') Both parties share responsibility – Admin splits costs proportionally.

Describing Use Cases [[Agreement Lock-in](#)]

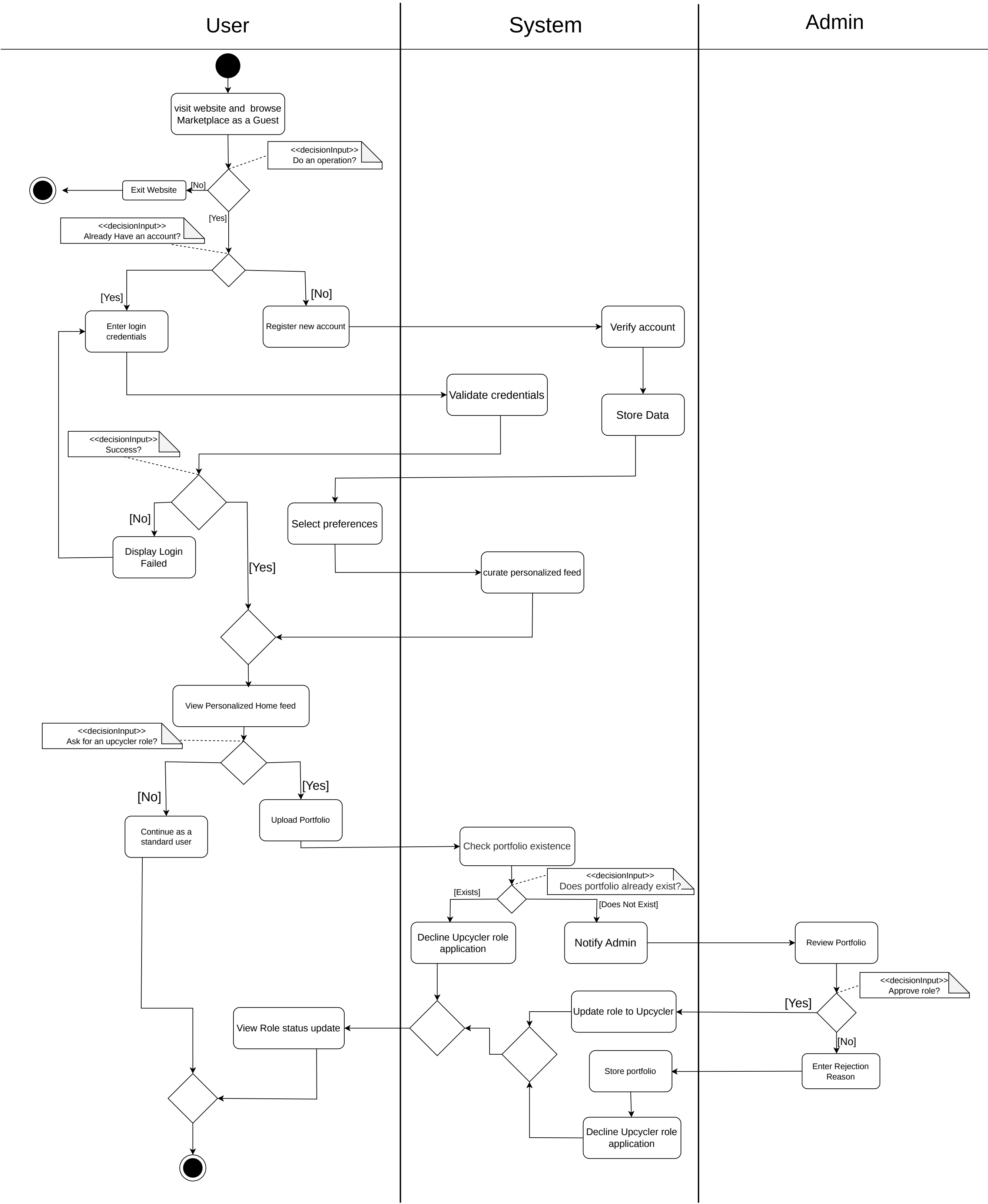
ID Number – Name:	UC-21 – Agreement Lock-in
Short-Description / Goal:	Once both parties (buyer/seller or two swappers) reach a final agreement, the system automatically freezes the item description, price, and photos to prevent any post-agreement modifications.
Preconditions:	Both parties have agreed to a transaction (sale or swap). Transaction is in 'Pending Confirmation' state.
Postconditions:	Item description, price, and photos are locked and cannot be edited. Both parties are notified that the agreement is locked. Transaction proceeds to payment/shipping phase.
Error Situations – System State in the Event of an Error:	Lock-in system fails – Transaction is paused and both parties are notified to re-confirm before proceeding.
Actor(s) / Initiator(s):	System (automated), Registered User (Buyer/Seller/Swapper)
Trigger:	Both parties digitally confirm the transaction agreement.
Standard Process (Main Success Scenario):	(1) Both parties confirm agreement (Buyer clicks 'Confirm Purchase' or both Swappers accept the swap). (2) System sets item status to 'Agreement Locked'. (3) System freezes all editable fields: description, photos, price, and swap terms. (4) Any subsequent edit attempts by either party are blocked. (5) System notifies both parties that the agreement is now locked. (6) Transaction proceeds to the payment or shipping phase.
Alternative Processes (Alternative/Other Scenarios):	(4') A party attempts to edit a locked field – System displays a 'Field Locked – Agreement in Progress' message.

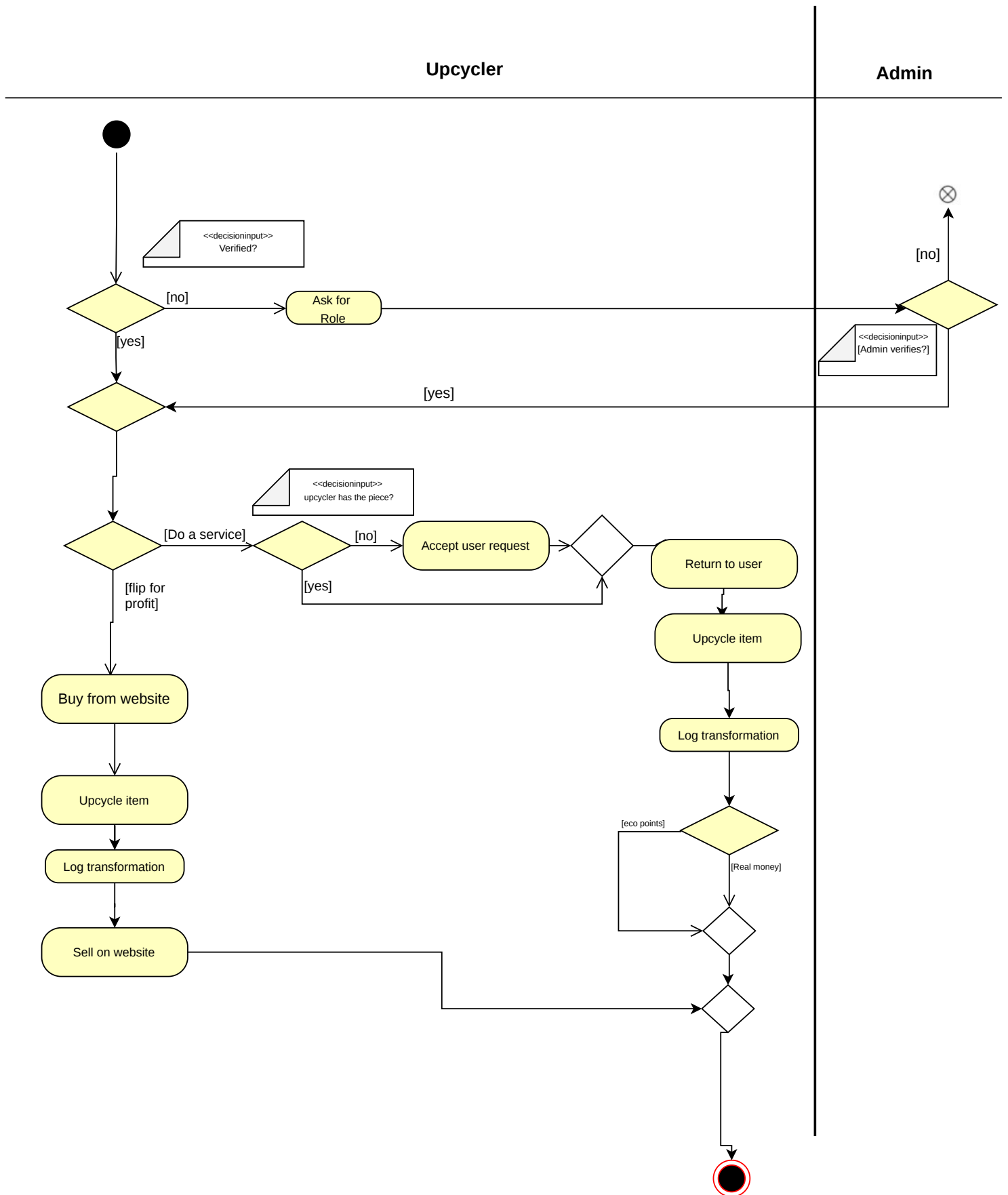
Describing Use Cases [[Dynamic Sync](#)]

ID Number – Name:	UC-22 – Dynamic Sync
Short-Description / Goal:	When an item enters an active agreement (sale or swap confirmed by both parties), the system automatically locks the item and removes it from the marketplace instantly, preventing it from appearing in any other listings or search results.
Preconditions:	Item is listed on the marketplace for sale or swap. Both parties have reached a confirmed agreement.
Postconditions:	Item is locked and no longer visible on the marketplace. Any other pending offers on the item are automatically cancelled and their initiators notified.
Error Situations – System State in the Event of an Error:	Sync fails and item remains visible in another listing – System raises an alert and re-triggers the sync; auto-cancels all pending offers on the item
Actor(s) / Initiator(s):	System (automated)
Trigger:	Both parties confirm a transaction agreement
Standard Process (Main Success Scenario):	(1) Both parties confirm the agreement. (2) System instantly locks the item. (3) Item is removed from the marketplace and no longer appears in search or browse results. (4) All other pending offers for the item are automatically cancelled. (5) Affected users (those who had pending offers) are notified of the cancellation. (6) Item status is updated to 'In Transaction' and hidden from the marketplace.
Alternative Processes (Alternative/Other Scenarios):	(4') No other pending offers exist — System skips cancellation step and proceeds directly.

Describing Use Cases [[Swap Meet Finder](#)]

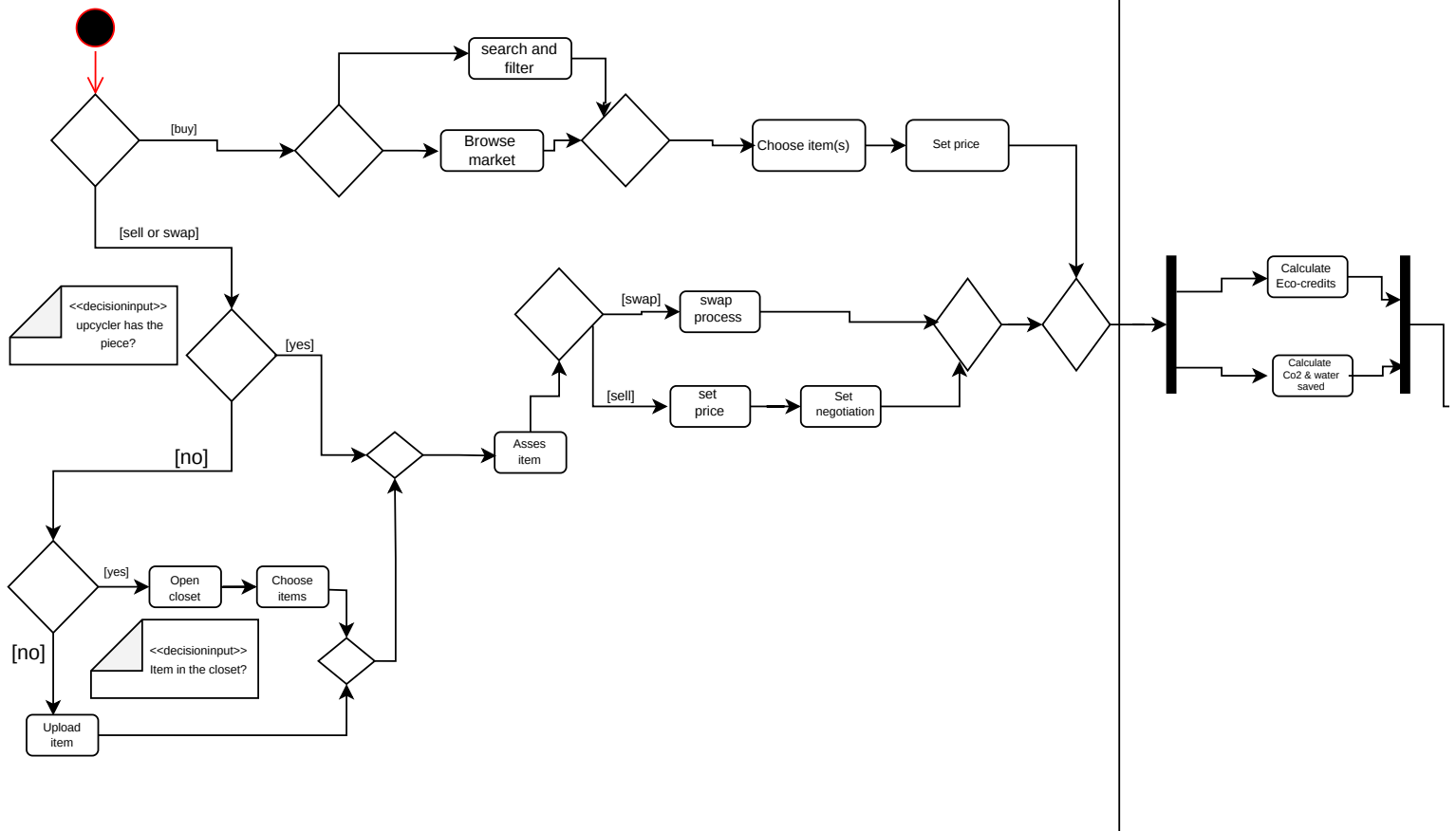
ID Number – Name:	UC-22 – Meet Finder
Short-Description / Goal:	The system identifies nearby users based on both parties' locations and, when the distance is within a defined threshold, suggests an in-person meetup for a swap instead of shipping — eliminating shipping emissions. The marketplace also prioritizes showing listings from users who are geographically close to the browsing user.
Preconditions:	Both users have granted location access. A swap offer is in progress, OR a user is browsing the marketplace.
Postconditions:	[Swap] Both parties arrange an in-person exchange; no shipping is generated. [Browse] User sees nearby listings ranked higher in results. Carbon savings from avoided shipping are calculated and credited.
Error Situations – System State in the Event of an Error:	Location data is unavailable for one or both users — System defaults to standard shipping flow and notifies users that Meet Finder is unavailable.
Actor(s) / Initiator(s):	System (automated), Registered User
Trigger:	A swap offer is initiated between two users, OR A user opens the marketplace (for proximity-based ranking).
Standard Process (Main Success Scenario):	(1) Swap offer is initiated between Swapper A and Swapper B. (2) System calculates the distance between both users' locations. (3) Distance is within the defined threshold — System suggests a local meetup instead of shipping. (4) Both parties agree to meet in person. (5) System marks the swap as "Local Meetup" — no shipping label is generated. (6) System marks the swap as complete and calculates carbon savings from avoided shipping.
Alternative Processes (Alternative/Other Scenarios):	(3') Distance exceeds the threshold — System proceeds with the standard shipping flow. (4') One party declines the meetup — System falls back to standard shipping.

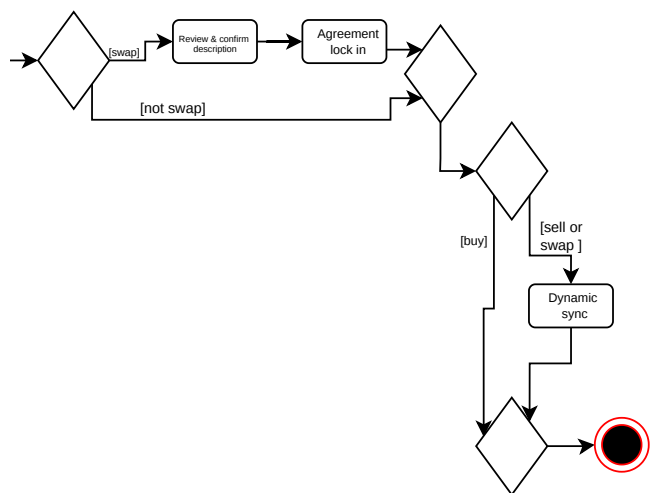


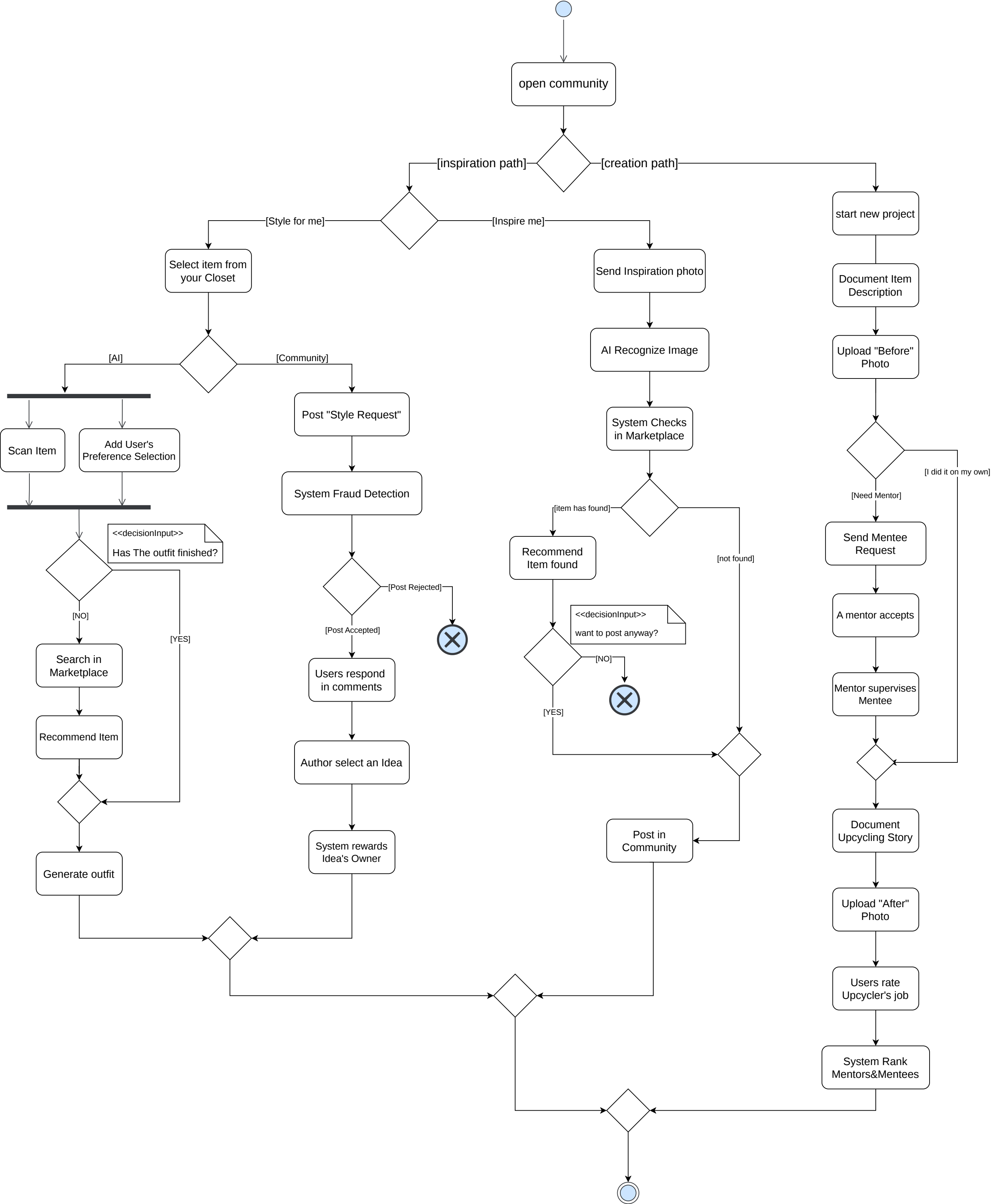


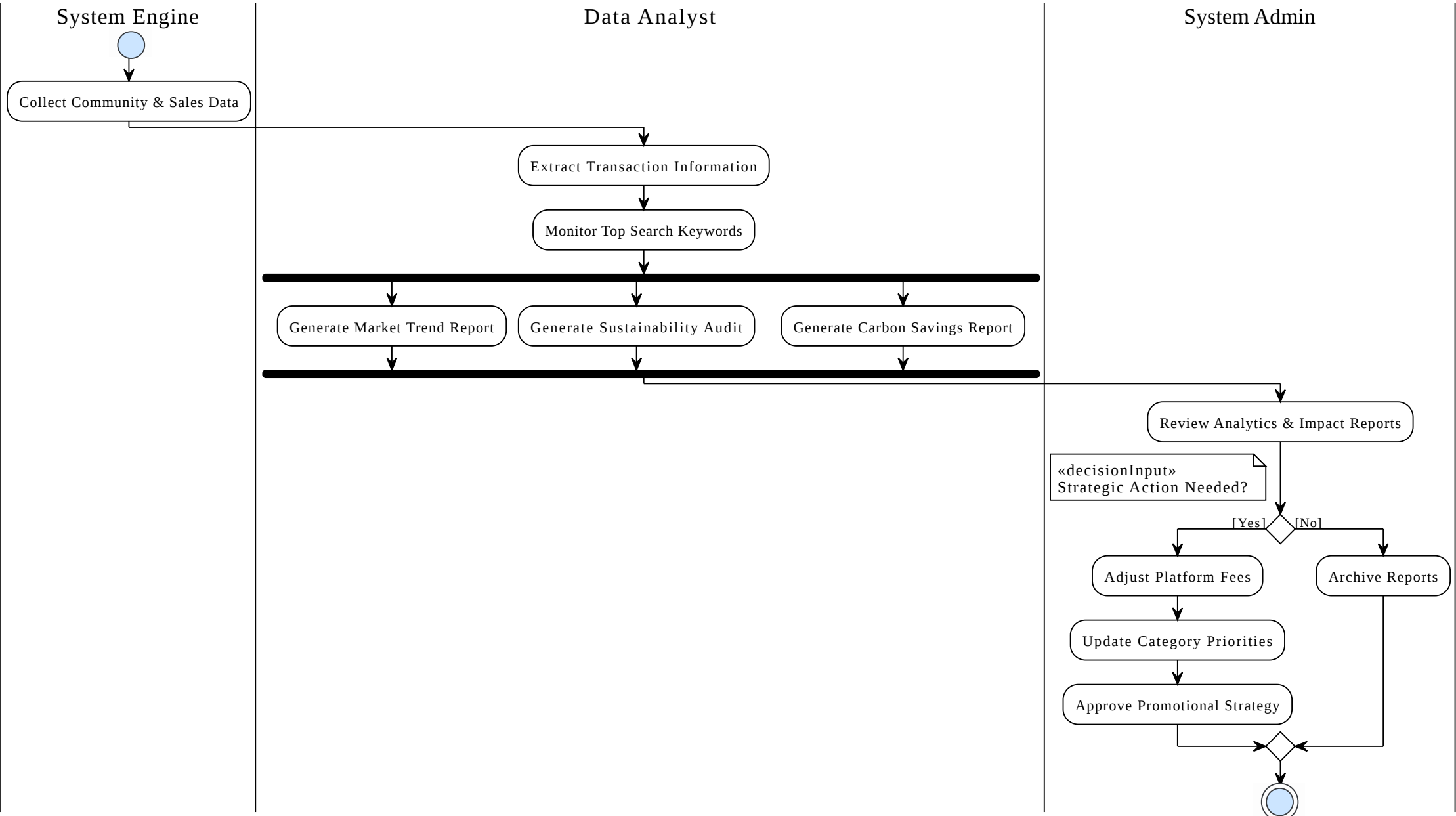
Registered User

System

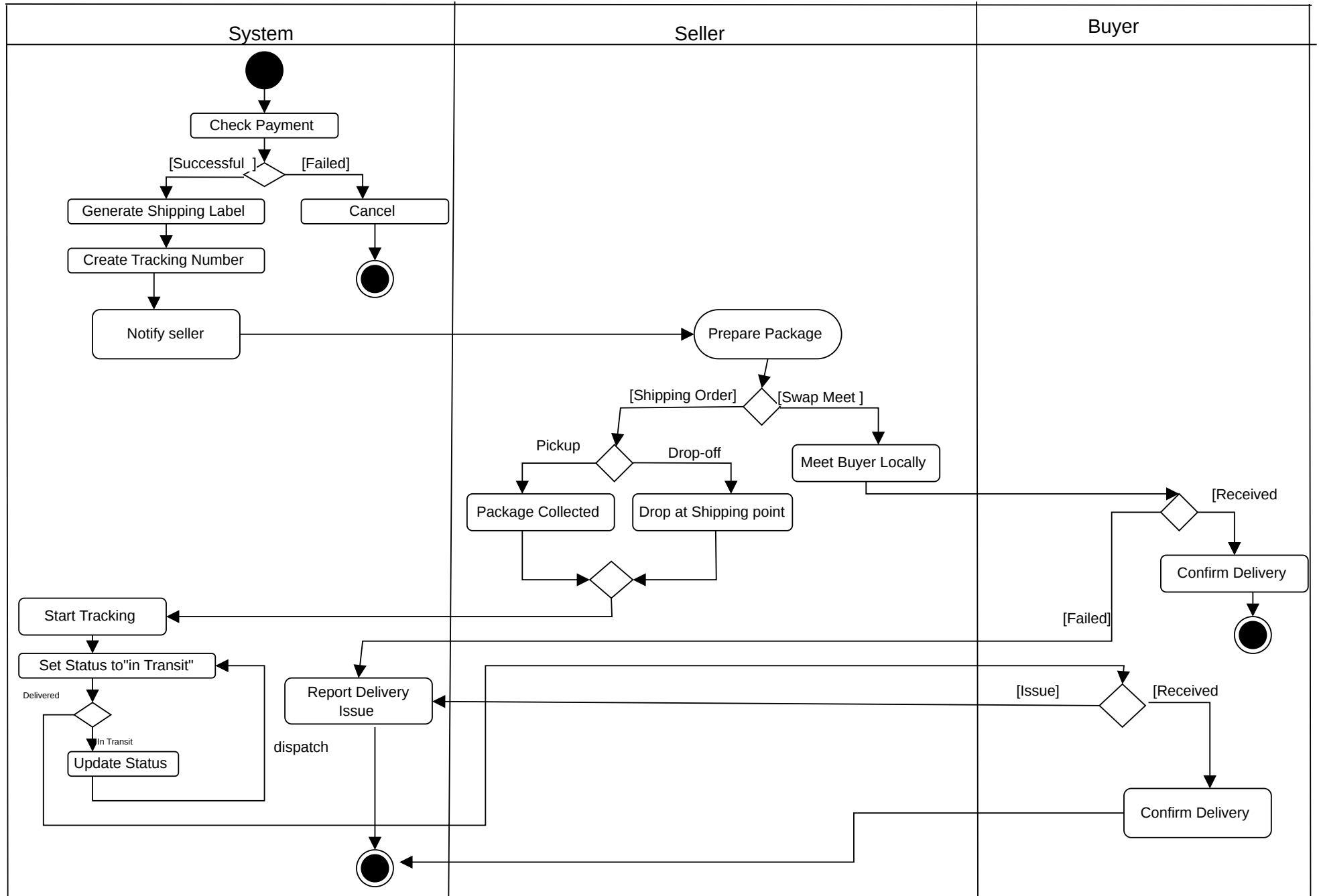


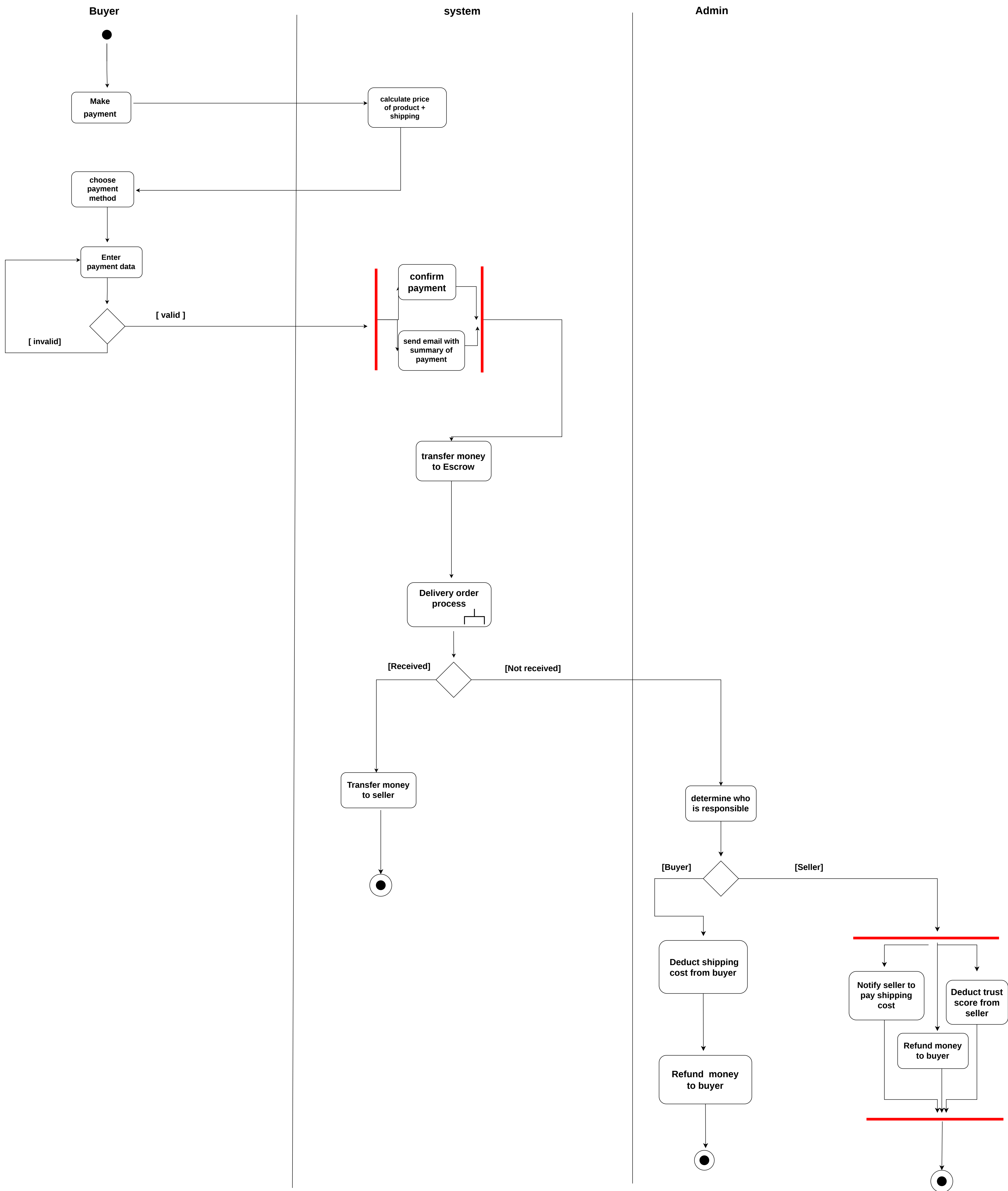


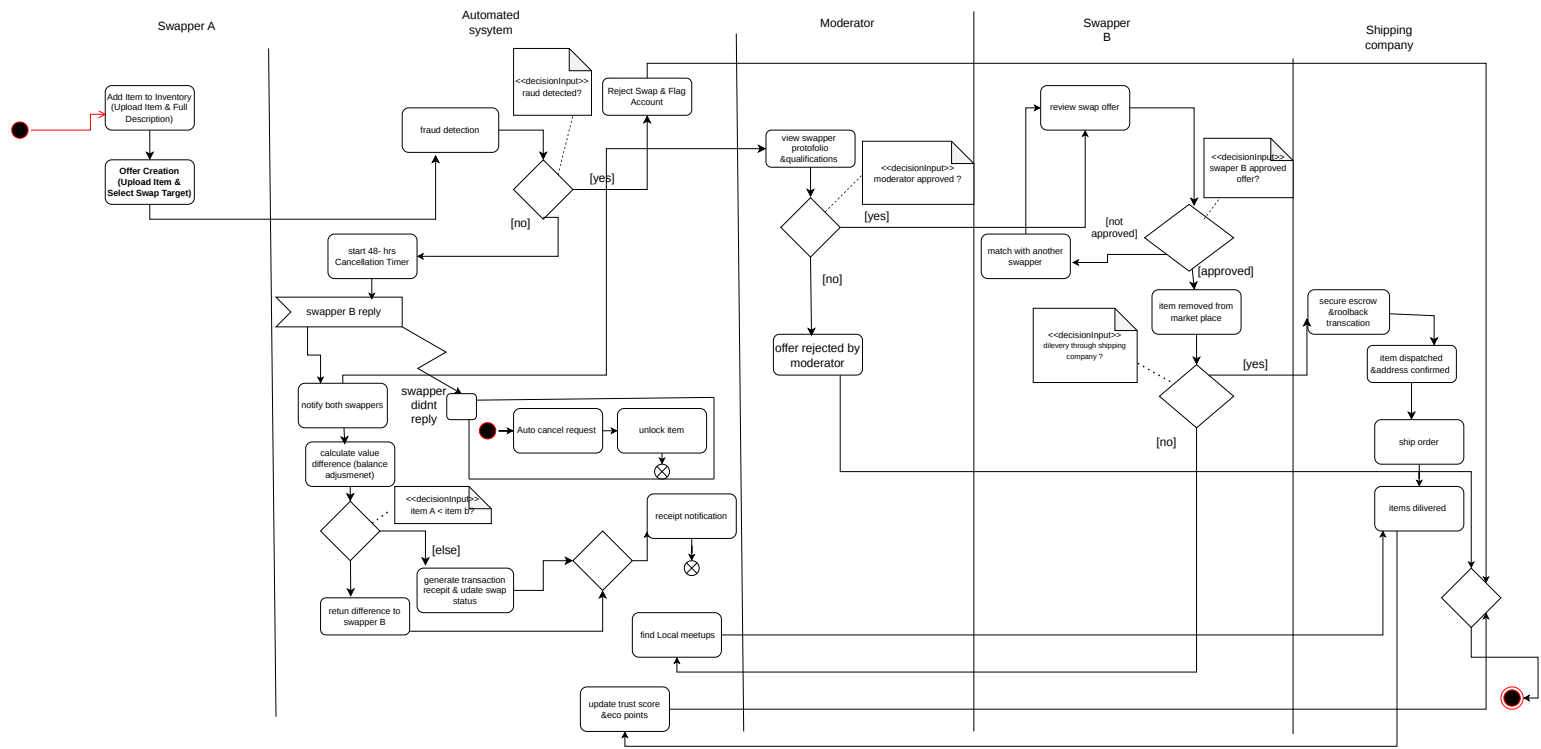


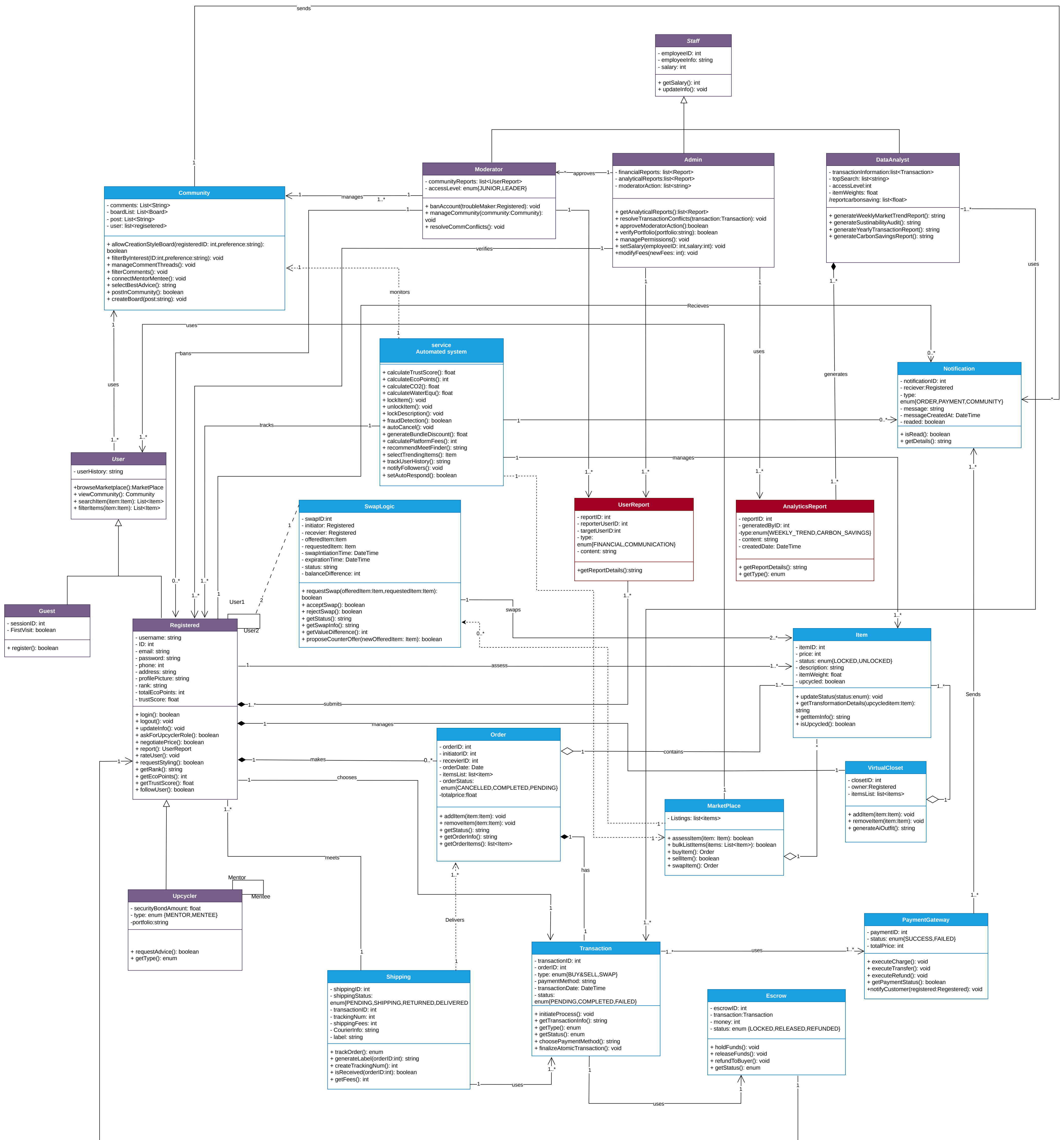


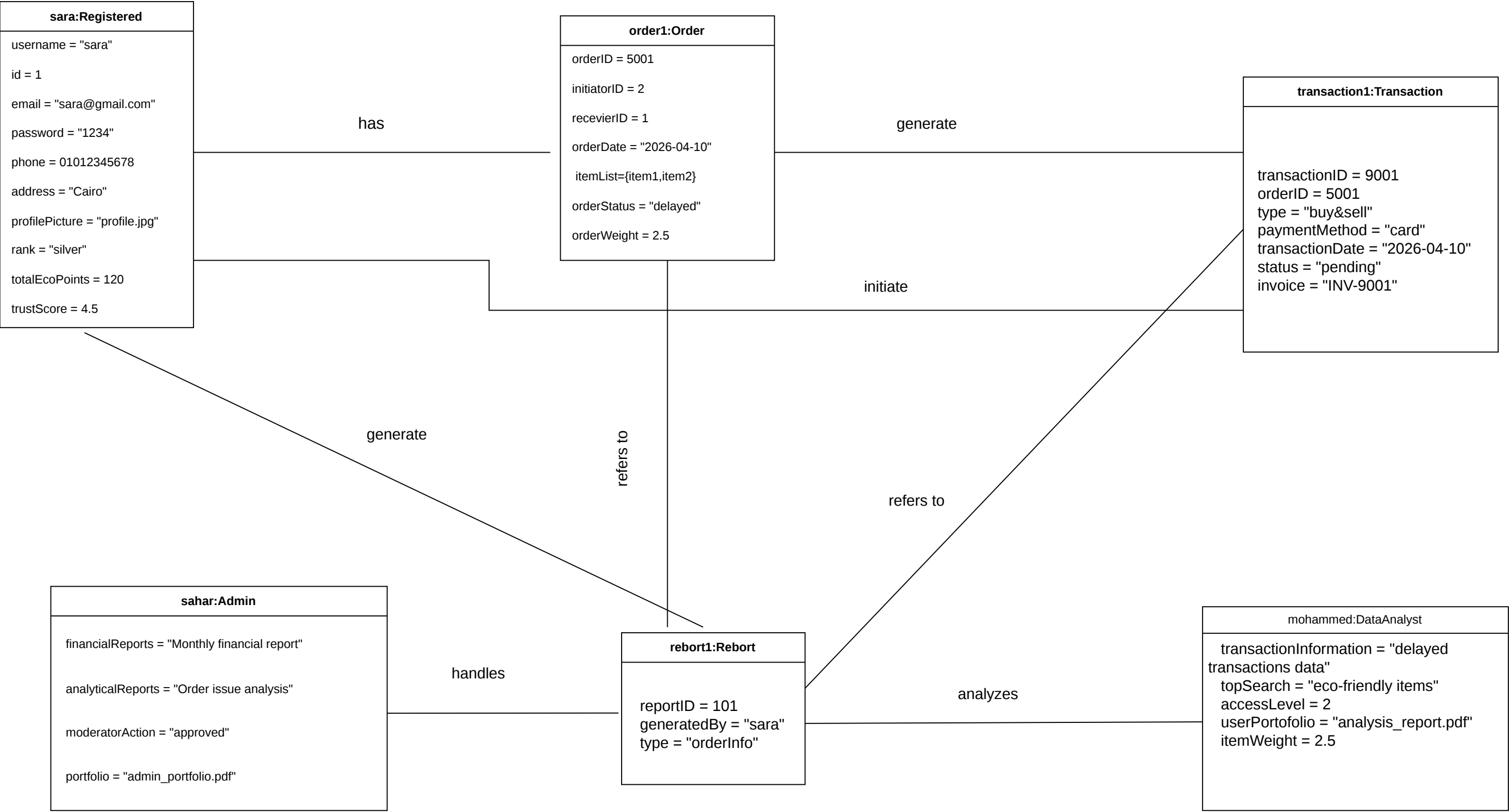
Delivery order process

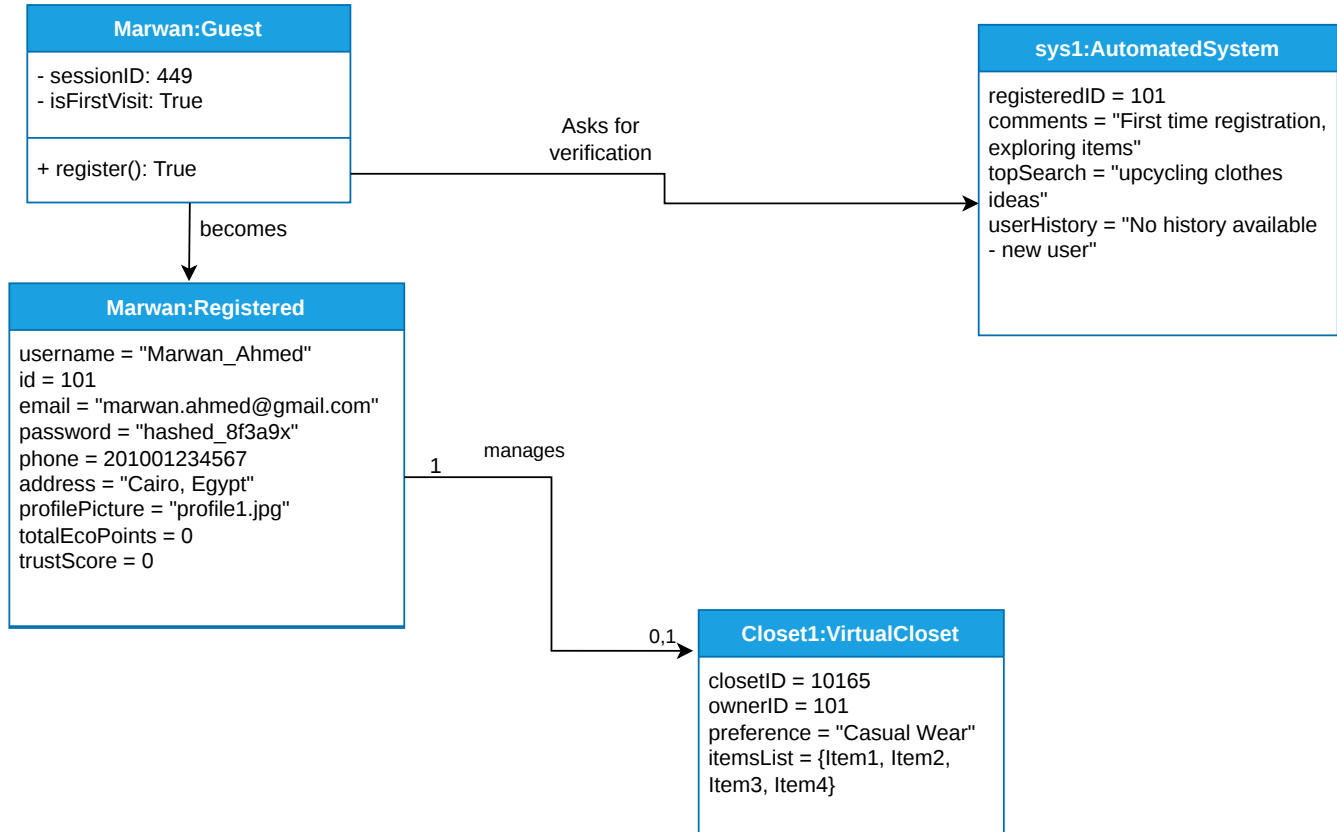












sara : Registered
username = "sara_eco"
id = 1001
email = "sara@eco.com"
password = "1234"
phone = "01012345678"
address = "Cairo, EG"
profilePicture = "sara.jpg"
rank = "Gold"
totalEcoPoints = 320
trustScore = 4.7

ahmed : Registered
username = "ahmed_up"
id = 1002
email = "ahmed@eco.com"
password = "02155"
phone = "01098765432"
address = "Alexandria, EG"
profilePicture = "ahmed.jpg"
rank = "Silver"
totalEcoPoints = 180
trustScore = 4.2

ecoMarket : MarketPlace
itemsList = [vintageJacket]
operation = [buy, sell, swap]
itemID = 5001
fees = 15

vintageJacket : Item
itemID = 5001
price = 350
status = "available"
description = "Vintage denim jacket"
itemWeight = 0.8
upcycled = true

notif1 : Notification
notificationID = 9001
recieverID = 1001
type = "payment"
message = "Purchase confirmed"
messageDate = 2025-04-14

notif2 : Notification
notificationID = 9002
recieverID = 1002
type = "order"
message = "Price offer: 300 EGP"
messageDate = 2025-04-14

receives

negotiates

negotiates

receives

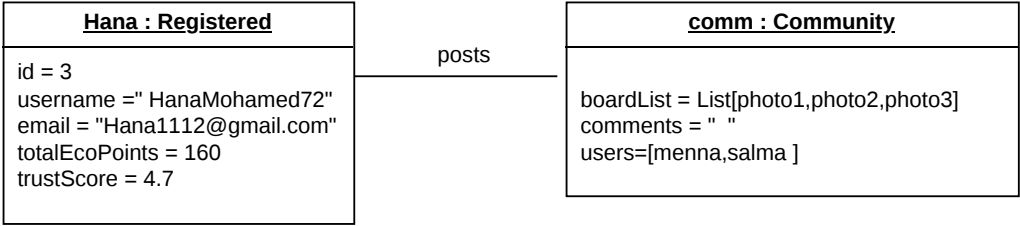
triggers

lists

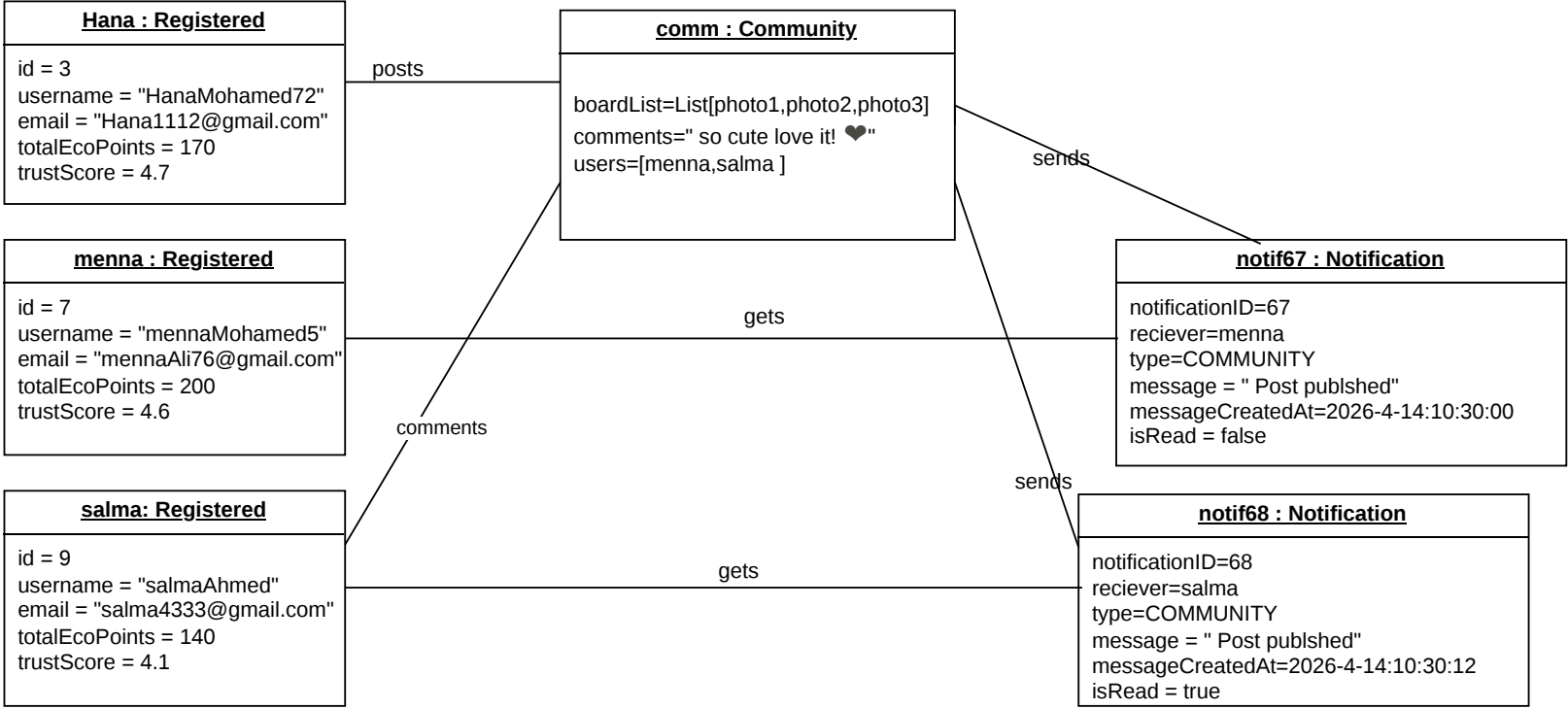
triggers

" Creat Board "

Pre-Condition:

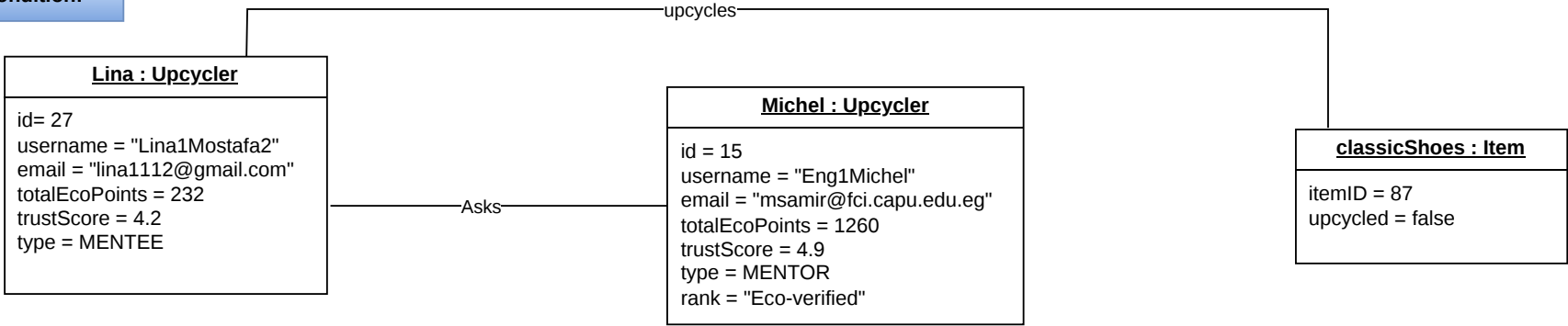


Post-Condition:

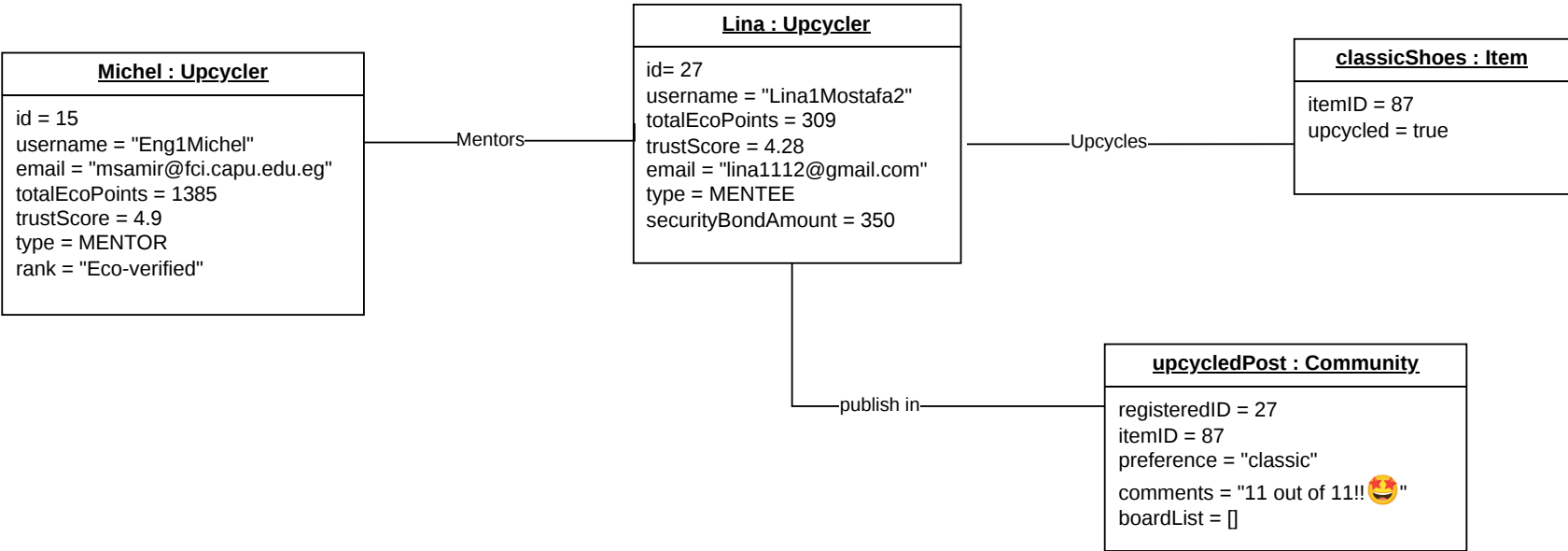


" Mentor A Mentee "

Pre-Condition:

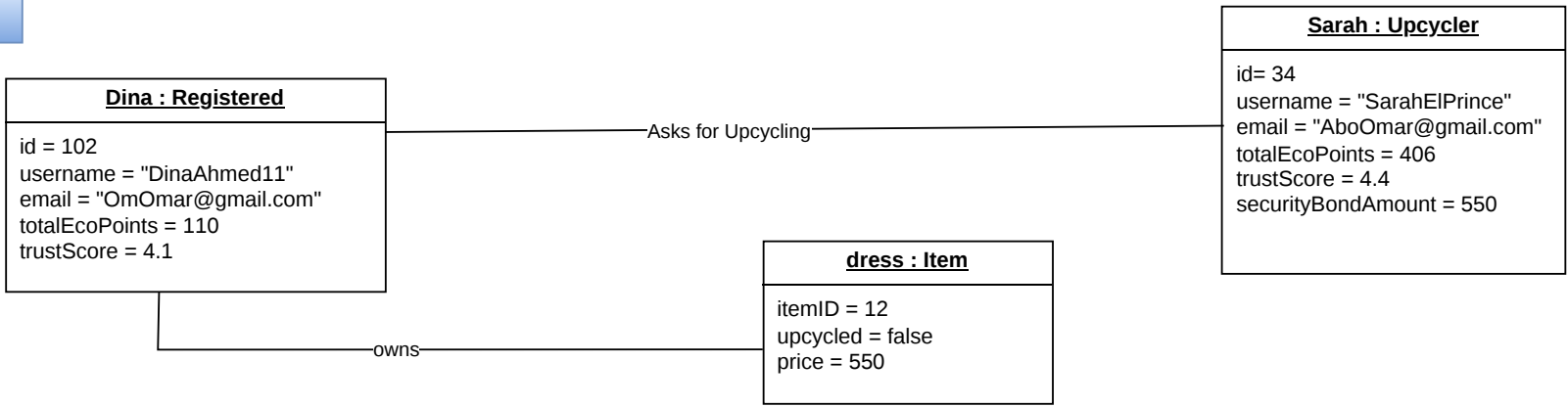


Post-Condition:

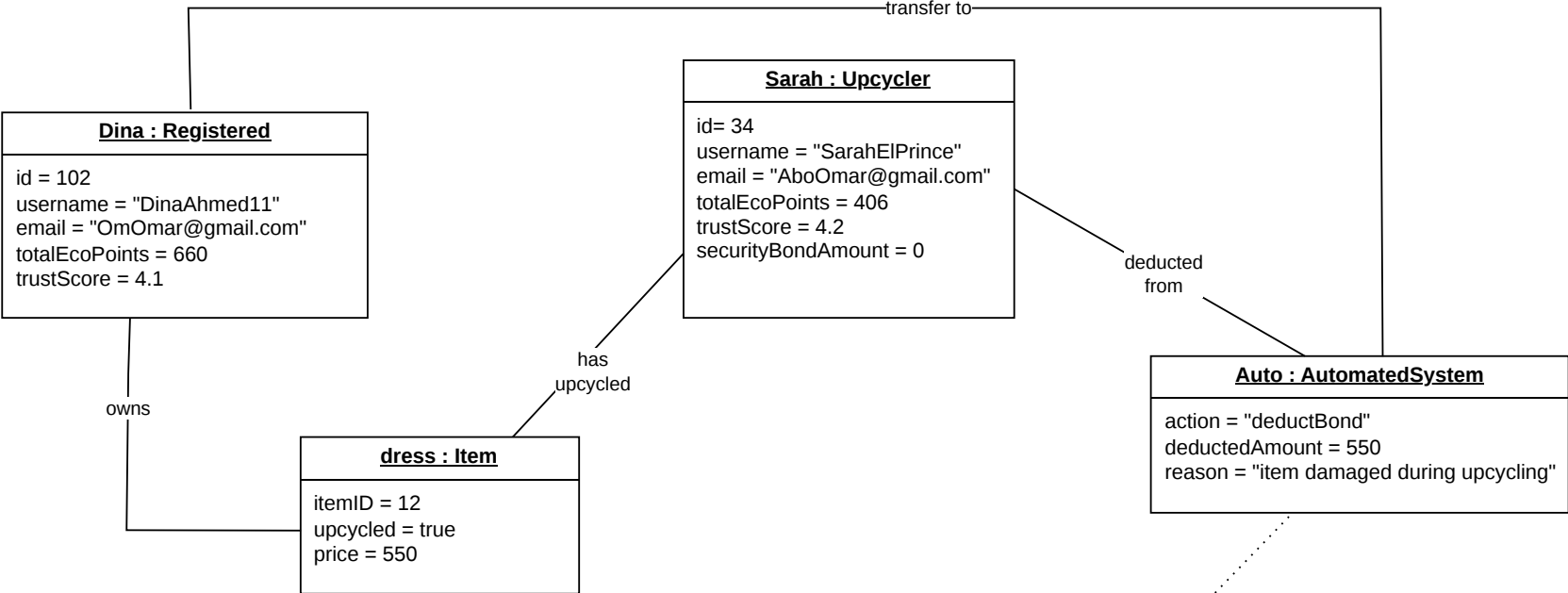


" Do A Service "

Pre-Condition:



Post-Condition:



"Upcycler failed to return item in acceptable condition, So System reduced his trust Score. Security bond of 550 EGP deducted and compensated via platform to item owner (Sarah)."

